

# 融合方案：客户-产品-库存联动决策系统

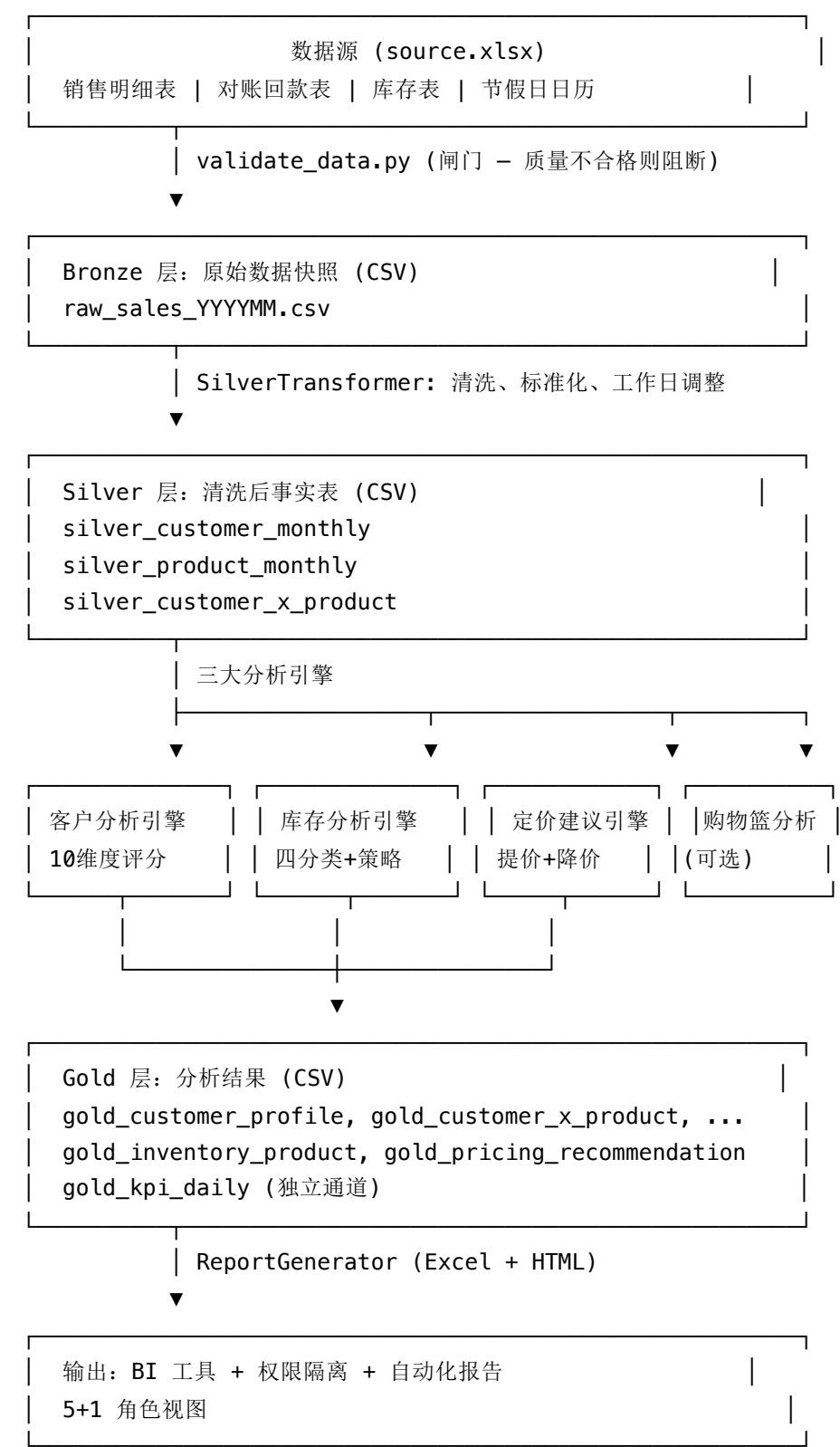
文档状态：v2.0 — 全面审查修订版  
目标读者：数据工程师、数据分析师、业务决策者  
阅读指南：

- 第一遍通读 **核心融合原则** + **架构总览** + **设计决策记录**
- 第二遍聚焦与你角色相关的实现层
- 每段代码上方有 **设计理由** 区块，解释为什么这样写

## 核心融合原则

| 原则  | 内容                                      | 来源     |
|-----|-----------------------------------------|--------|
| 数据层 | Medallion 架构（Bronze → Silver → Gold）为底座 | PDF 方案 |
| 计算层 | Python 为主，VBA 仅用于 Excel 输出格式化           | 技术选型共识 |
| 应用层 | 三层权限隔离 + 五角色视图 融合为统一视图体系                | 融合两方案  |
| 扩展点 | 库存联动、降价模拟、购物篮分析作为增强模块，主流程不依赖            | 架构决策   |

# 架构总览



# 各层职责

| 层       | 职责                | 输入          | 输出             | 运行频率 |
|---------|-------------------|-------------|----------------|------|
| Bronze  | 原始数据快照，保留现场       | source.xlsx | raw CSV        | 每次运行 |
| Silver  | 清洗、标准化、工作日调整、买赠还原 | Bronze CSV  | 3张 Silver 表    | 每次运行 |
| Gold    | 分析计算、评分、分类        | Silver 表    | 6张 Gold 表      | 每次运行 |
| 准实时 KPI | 日更轻量聚合            | 最新导出        | gold_kpi_daily | 每日   |

# 设计决策记录 (Architecture Decision Record)

每项决策说明「为什么选 A 而非 B」，帮助接手者理解上下文。

## ADR-001：Medallion 架构

- **背景：**两方案分别采用「直接分析」和「分层加工」
- **决策：**采用 Medallion Bronze → Silver → Gold
- **理由：**
  - Bronze 保留原始快照，可在不重新导出数据的情况下回溯
  - Silver 一次清洗，Gold 多个引擎复用同一套干净数据
  - 某层出错只需重刷该层，不需要全量重跑
- **不选：**直接分析方案 —— 无法追踪数据血缘，问题难定位

## ADR-002：Python 为主，VBA 为辅

- **决策：**清洗和分析用 Python，VBA 仅 Excel 格式化
- **理由：**
  - Python 生态：pandas（表格处理）、scipy（统计）、mlxtend（关联规则）、plotly（可视化）
  - VBA 无法处理 >10 万行数据，无法集成 scikit-learn
  - VBA 仍然用于 Excel 输出格式定制，因为 openpyxl 的精细格式控制成本较高
- **条件失效：**团队完全无 Python 能力且拒绝学习 → 降级为纯 VBA 方案（功能缩水 60%）

## ADR-003：订单编号可配置

- **背景：**源数据中订单号字段名可能为 订单编号 、 OrderID 、 销售订单 、 单据号
- **决策：**在 config.py 的 COLUMN\_MAPPING 中统一配置，支持别名自动匹配
- **理由：**不同公司的 ERP 系统字段命名不同，硬编码会导致每次对接新数据源都要改代码
- **使用方式：**如果数据源无订单编号（如仅汇总数据），设 ALLOW\_EMPTY\_ORDER\_ID = True ，系统会跳过依赖订单编号的功能

## ADR-004：工作日调整需要同时保留原始值和调整值

- **背景：**文字版要求工作日调整，PDF 方案未涉及
- **决策：**在 Silver 层生成调整值，Gold 层默认使用调整值，但 Gold 表保留原始值做对照
- **理由：**
  - 环比分析需要消除月份工作日差异（2月 vs 3月）
  - 财务报告需用原始值（实际开票金额）
  - 同时保留两套值让分析师可自行选择口径

## ADR-005：参考日期固定而非动态

- **背景：**原方案用 `pd.to_datetime('now')` 计算「距上次采购天数」
- **决策：**使用数据中最大日期或 `config.analysis_reference_date` 作为参考
- **理由：**每日运行结果不同会破坏审计追溯，无法回答「上周五的报告结果是多少」

# 第一层：基础数据验证与框架搭建（第 1-2 周）

## 1.1 环境准备与依赖安装

# 基础环境

Python 3.9+

```
pip install pandas>=2.0 numpy>=1.24 openpyxl>=3.1 plotly>=5.18 scipy>=1.11 scikit-learn>=1.3
```

# 购物篮分析（可选高级模块）

```
pip install mlxtend
```

## 1.2 目录结构（融合版）

```
customer_inventory_linkage/
├── config.py                # 统一配置（融合两方案参数）
├── data/
│   ├── source.xlsx         # 原始数据（必须包含的工作表见 1.4）
│   └── holidays.xlsx       # 节假日日历（工作日调整用）
├── bronze/                 # 原始数据层快照
│   └── raw_sales_YYYYMM.csv
├── silver/                 # 清洗认证层
│   ├── silver_customer_monthly.csv
│   ├── silver_product_monthly.csv
│   └── silver_customer_x_product.csv
├── gold/                   # 分析结果层
│   ├── gold_customer_profile.csv
│   ├── gold_customer_x_product.csv
│   ├── gold_customer_portfolio_health.csv
│   ├── gold_inventory_product.csv
│   ├── gold_kpi_daily.csv
│   └── gold_pricing_recommendation.csv
├── output/
│   ├── reports/            # Excel + HTML 报告
│   └── bi_ready/           # BI 工具直接消费的宽表
└── scripts/
    ├── run_pipeline.py     # 主流程编排
    ├── run_kpi_daily.py    # 准实时 KPI 通道
    └── validate_data.py    # 数据质量验证
```

## 1.3 配置文件（[config.py](#)）

### 设计理由：

- 所有阈值集中管理，不硬编码在分析代码中
- COLUMN\_MAPPING 支持字段名自动匹配，对接不同 ERP 系统时只需修改此处
- 订单编号 字段名可配置，缺省时可跳过订单级分析
- ANALYSIS\_REFERENCE\_DATE 固定时间锚点，保证每次运行结果可复现
- ETL\_RUN\_ID 用于追踪每次运行的审计线索

"""

统一配置 - 融合 PDF 方案和文字版方案的所有阈值与映射

使用方式:

```
from config import config
workday_cfg = config['CLEANING_RULES']['workday_adjustment']
```

新人须知:

1. 初次部署时, 根据源数据实际字段名修改 COLUMN\_MAPPING
2. 所有阈值参数可在试运行后根据业务反馈调整
3. 不要在此文件中写业务逻辑

"""

```
import os
import uuid
from datetime import datetime
from pathlib import Path

# =====
# 运行环境
# =====
ETL_RUN_ID = str(uuid.uuid4())[:8]          # 每次运行生成唯一 ID, 用于审计追踪
ETL_RUN_AT = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

# =====
# 路径配置
# =====
BASE_DIR = Path(__file__).parent
DATA_DIR = BASE_DIR / 'data'
BRONZE_DIR = BASE_DIR / 'bronze'
SILVER_DIR = BASE_DIR / 'silver'
GOLD_DIR = BASE_DIR / 'gold'
OUTPUT_DIR = BASE_DIR / 'output'

# =====
# 数据字段映射
# 设计理由: 源数据字段名因 ERP 供应商而异, 此处统一配置别名
# 使用方式: 键为内部标准名, 值为可能出现的源数据列名列表
# =====
COLUMN_MAPPING = {
    '客户编号':      ['客户编号', 'CustomerID', '客户ID', '客户代码'],
    '客户名称':      ['客户名称', 'CustomerName', '客户全称'],
    '渠道类型':      ['渠道类型', 'Channel', '直销/代理', '销售渠道'],
    '产品品种':      ['产品品种', 'ProductCode', '料号', 'SKU', '产品编码'],
    '产品一级分类':  ['产品一级分类', 'ProductLine', '产品线', '产品大类'],
    '产品二级分类':  ['产品二级分类', 'ProductSubLine', '产品子类'],
    '发货日期':      ['发货日期', 'DeliveryDate', '实际发货日期', '出库日期'],
```

```

'下单日期':      ['下单日期', 'OrderDate', '订单日期'],
'发货数量':      ['发货数量', 'Quantity', '数量', '出库数量'],
'单价':          ['单价', 'UnitPrice', '不含税单价', '含税单价'],
'金额':          ['金额', 'Amount', '行金额', '不含税金额', '销售额'],
'成本':          ['成本', 'Cost', '标准成本', '单位成本'],
'业务负责人':    ['业务负责人', 'SalesRep', '销售员', '销售人员', '销售'],
'代理商名称':    ['代理商名称', 'Agent', '代理渠道', '经销商'],
# 订单编号可配置：字段名因 ERP 不同而异
# 如果数据源无订单编号，将 ALLOW_EMPTY_ORDER_ID 设为 True
'订单编号':      ['订单编号', 'OrderID', '销售订单', '单据号', '订单号', 'SoNumber'],
}
# 是否允许订单编号为空（数据源无订单号时设为 True）
ALLOW_EMPTY_ORDER_ID = False

# =====
# 分析参考日期
# 设计理由：固定参考日期保证结果可复现。
# - 设为 'auto' 则自动使用数据中的最大日期
# - 设为具体日期字符串则锁定在该日期
# - 不要用 pd.to_datetime('now')，否则每次跑结果不同
# =====
ANALYSIS_REFERENCE_DATE = 'auto' # 或 '2026-05-18'

# =====
# 清洗规则（融合两方案）
# =====
CLEANING_RULES = {
    # PDF 方案：负销量过滤（退货在 match_returns 中处理，此处仅过滤无效负值）
    'filter_negative_sales': True,

    # PDF 方案：毛利率 Winsorization（极端值钳制）
    'margin_winsorization': {
        'enabled': True,
        'lower_clip': -0.50, # 毛利率低于 -50% 的钳制到 -50%
        'upper_clip': 0.75, # 高于 75% 的钳制到 75%
        # 设计理由：低于 -50% 通常是数据错误（如成本>售价2倍）；
        # 高于 75% 可能是买赠导致的极端值，实际业务极少持续 >75% 毛利率
    },

    # 文字版：退货冲销（后进先出匹配）
    'return_matching': {
        'enabled': True,
        'match_window_days': 90, # 90 天内匹配原始销售
        'match_dimensions': ['客户编号', '产品品种'],
        # 设计理由：退货通常发生在发货后的 1-2 个季度内；
        # 超过 90 天可能是质保退货或换货，应单独处理
    },
}

```

```

# 文字版：买赠还原（按数量比例分摊实收金额）
'promotion_allocation': {
    'enabled': True,
    'identify_by': ['业务负责人', '代理商名称'],
    'allocation_method': '按数量比例分摊',
    # 设计理由：同一业务员-代理商-日期窗口内的多产品交易，
    # 买一赠一或打包销售导致单品实收为 0，按数量分摊还原真实单价
},

# 文字版：工作日调整
'workday_adjustment': {
    'enabled': True,
    'standard_workdays_per_month': 21,    # 标准月工作日
    'min_valid_workday_ratio': 0.70,      # 当月工作日 < 标准的 70% 时标记为无效
    'holidays_file': 'data/holidays.xlsx',
    # 设计理由：2 月（约 20 天）和 3 月（约 23 天）的自然销量差异可达 15%，
    # 不做调整会导致 2 月「异常下滑」的误判
},

# 文字版：项目备货标记
'project_stock_flag': {
    'enabled': True,
    'threshold_multiplier': 3,              # 单次采购量 > 前12月月均量 × 3
    'extend_warning_window': True,         # 拉长后续采购间隔预警窗口
},
}

# =====
# 客户分层参数（融合两方案）
# =====
CUSTOMER_SEGMENTATION = {
    # PDF 方案：RFM- $\pi$  评分权重（R=最近购买，F=频率，M=金额， $\pi$ =利润）
    'rfm_pi_weights': {
        'R': 0.25,
        'F': 0.25,
        'M': 0.25,
        'pi': 0.25,
    },

    # 文字版：KA/AA 标记（如果数据中有则优先使用）
    'ka_aa_priority': True,

    # 文字版：非重点客户分层阈值
    'non_ka_segmentation': {
        'high_potential': {
            'sales_12m_min_percentile': 0.50,
            'growth_6m_min': 0.10,
        },
    },
}

```



```

        'low_value': {
            'sales_12m_max_percentile': 0.20,
            'growth_6m_max': 0,
        },
    },

# 流失预警阈值
'churn_warning': {
    'suspected_days': 90,    # >90 天无采购 → 疑似流失
    'high_risk_days': 180,   # >180 天 → 高危
},
}

# =====
# 信用评分参数（PDF 方案核心）
# =====
CREDIT_SCORING = {
    'dso_buckets': {
        'A': {'max_dso': 45, 'max_overdue_rate': 0},
        'B': {'max_dso': 60, 'max_overdue_rate': 0.05},
        'C': {'max_dso': 90, 'max_overdue_rate': 0.15},
        'D': {'max_dso': float('inf'), 'any_bad_debt': True},
    },
    'payment_pattern': {
        'track_installments': True,
        'cluster_method': 'kmeans',
    },
}

# =====
# 产品生命周期参数（融合两方案）
# =====
PRODUCT_LIFECYCLE = {
    # PDF 方案：品种状态机
    'sku_state_machine': {
        'new': '该品种在该客户首次出现于当月',
        'continuous': '上月和当月均有发货',
        'disappeared': '上月有、当月无发货',
        'revived': '消失至少 1 个月后再次出现',
        'surge': '当月数量较上月增长超过 100%',
        'decline': '连续 3 个月数量下降且当月下降超过 30%',
    },

    # 文字版：客户-产品生命周期阶段
    'customer_product_stage': {
        'introduction': {
            'condition': '首次购买且当日采购数量 > 6000',
            'description': '大批量导入',
        }
    }
}

```

```

    },
    'ramp_up': {
        'condition': '近 3 月调整月销量环比增长  $\geq 15\%$ ',
        'min_months_after_intro': 1,
    },
    'mature': {
        'condition': '近 6 月调整月销量在均值  $\pm 1.5$  倍标准差内',
        'season_adjusted': True,
    },
    'decline': {
        'condition': '连续 3 月调整销量低于季节校正后均线 15% 以上, '
        '或距上次采购 > 2 倍常规间隔',
        'exclude_project_stock': True,
    },
},
}

```

```

# =====

```

```

# 库存分类参数（文字版核心）

```

```

# =====

```

```

INVENTORY_CLASSIFICATION = {
    'fast_moving': {
        'max_turnover_days': 30,
        'min_active_customers': 3,
        'strategy': '常规安全库存监控',
    },
    'dormant_activatable': {
        'min_turnover_days': 90,
        'has_shipment_6m': True,
        'has_historical_customers': True,
        'strategy': '阶梯折扣 + 互补品打包, 清库销售额不计入个人考核',
    },
    'dormant_risk': {
        'min_turnover_days': 180,
        'no_shipment_12m': True,
        'has_no_customers': True,
        'strategy': '根因分析（客户单一 / 需求误判 / 技术退市）',
    },
    'growth_reserve': {
        'min_growth_3m': 0.20,
        'min_active_customers': 2,
        'max_stock_coverage_days': 14,
        'strategy': '动态安全库存, 缺货预警',
    },
    'single_customer_high_volatility': {
        'condition': '客户数 = 1 且近 6 月波动系数 > 0.5',
        'action': '采购申请时强制提示备货风险',
    },
}

```

```

}

# =====
# 定价与毛利参数（文字版核心，PDF 补充）
# =====
PRICING = {
    # 销售员不可见字段
    'sales_rep_hidden_fields': ['成本', '毛利率', '毛利额', '薪酬'],

    # 常规报价建议
    'price_uplift': {
        'compare_basis': '同类客户均价',
        'suggested_range_percentile': [0.25, 0.75],
    },

    # 降价增量策略（盈亏平衡计算）
    'markdown_strategy': {
        'trigger_conditions': [
            '产品处于成熟/衰退期',
            '客户历史有价格弹性',
            '降价后目标销量 ≥ 盈亏平衡销量的 1.2 倍',
            '新毛利率不低于公司当前整体水平',
        ],
        'formula': {
            'break_even_qty': '(P0 - C) * Q0 / (P1 - C)',
            'target_qty_constraint': 'Q_target >= 1.2 * Q_break',
            'total_margin_constraint': 'G1 > G0 and (G1 - G0) / G0 >= 0.10',
        },
        'elasticity': {
            'default': -1.0,          # 保守默认弹性系数
            'min_observations': 3,    # 至少 3 次价格变动才做拟合
            'fit_by': '产品品种',     # 按产品粒度拟合
            'clip_range': [-5.0, -0.1], # 弹性系数合理范围
        },
        'sales_rep_display': {
            'show': ['当前售价', '建议价格', '目标销量', '预计采购额变化', '销量底线'],
            'hide': ['成本', '盈亏平衡销量', '毛利率', '毛利额'],
        },
    },
}

# =====
# 购物篮分析参数（文字版高级功能）
# =====
MARKET_BASKET = {
    'enabled': True,
    'basket_definitions': {
        'order_basket': {'weight': 0.50, 'definition': '同一订单号内的产品'},
    }
}

```

```

        'time_basket': {'weight': 0.30, 'definition': '同一客户同一自然月内的产品'},
        'project_basket': {'weight': 0.20, 'definition': '某产品首次出现前后各 60 天内的产品'},
    },
    'association_rules': {
        'min_support': 0.01,
        'min_confidence': 0.25,
        'min_lift': 1.3,
    },
    'scoring_weights': {
        'weighted_confidence': 0.30,
        'lift': 0.20,
        'first_purchase_trigger': 0.20,
        'incremental_trigger': 0.15,
        'overall_coverage': 0.10,
        'ka_aa_coverage': 0.05,
    },
    'complementary_threshold': 60,
}

```

```

# =====
# 输出规范（融合两方案）
# =====

```

```

OUTPUT_SPEC = {
    'gold_customer_profile': {
        'granularity': '每客户一行',
        'columns': [
            '客户编号', '客户名称', '渠道类型', '客户等级',
            '近12月收入', '近12月毛利', '近12月毛利率', '收入增长率_同比',
            '收入增长率_环比', '连续增长月数', '连续下滑月数',
            '代理商名称', '终端客户', '渠道集中度',
            'DSO', '回款效率', '逾期率', '信用等级', '回款模式',
            '机会分', '机会等级', '机会主导因素',
            '风险分', '风险等级', '风险主导因素',
            'R得分', 'F得分', 'M得分', 'pi得分', 'RFM综合评分',
            '产品组合健康度', '高风险产品占比', '衰退品金额占比',
            '采购间隔预警', '产品集中度', '品类接受度',
            '建议动作1', '建议动作2', '建议动作3', '负责人',
            'etl_run_id', 'created_at',
        ],
    },
    'gold_customer_x_product': {
        'granularity': '每客户 × 每产品 × 月份一行',
        'columns': [
            '月份', '客户编号', '产品品种', '产品线',
            '月度金额', '月度数量', '毛利率', '采购份额',
            '产品状态', '生命周期阶段', '产品画像标签',
            '6因子风险评分', '是否新品',
            'etl_run_id', 'created_at',
        ],
    },
}

```

```

    ],
},
'gold_inventory_product': {
    'granularity': '每产品一行',
    'columns': [
        '产品品种', '产品线', '当前库存数量', '当前库存金额',
        '入库日期', '近3月调整日均销量', '库存周转天数',
        '库存分类', '活跃客户数', '近6月波动系数',
        '备货风险标记', '建议策略',
        'etl_run_id', 'created_at',
    ],
},
'gold_pricing_recommendation': {
    'granularity': '每销售员 × 每客户 × 每产品一行',
    'columns': [
        '业务负责人', '客户编号', '产品品种',
        '历史成交均价', '同类客户均价', '建议报价下限', '建议报价上限',
        '预计年度采购额变化', '策略类型',
        'etl_run_id', 'created_at',
    ],
},
},
}

# -----
# 快捷引用
# -----

config = {
    'COLUMN_MAPPING': COLUMN_MAPPING,
    'ALLOW_EMPTY_ORDER_ID': ALLOW_EMPTY_ORDER_ID,
    'ANALYSIS_REFERENCE_DATE': ANALYSIS_REFERENCE_DATE,
    'CLEANING_RULES': CLEANING_RULES,
    'CUSTOMER_SEGMENTATION': CUSTOMER_SEGMENTATION,
    'CREDIT_SCORING': CREDIT_SCORING,
    'PRODUCT_LIFECYCLE': PRODUCT_LIFECYCLE,
    'INVENTORY_CLASSIFICATION': INVENTORY_CLASSIFICATION,
    'PRICING': PRICING,
    'MARKET_BASKET': MARKET_BASKET,
    'OUTPUT_SPEC': OUTPUT_SPEC,
}

```

## 1.4 数据质量验证脚本 (validate\_data.py)

### 设计理由：

- 数据质量验证是整个 Pipeline 的第一道闸门

- 分为 CRITICAL（阻断级，不修复不允许进入分析）和 WARNING（警示级，标注后可放行）
- 验证报告独立输出 JSON + Excel，供下游程序自动判断（json）和人工审阅（excel）
- 每个验证方法返回独立的 DataFrame，最后由入口统一汇总，避免状态累积

"""

数据质量验证 - 融合两方案的红线检查

输出:

1. output/data\_validation\_report.json (机器可读, 下游 Pipeline 据此判断是否继续)
2. output/数据质量验证报告.xlsx (人肉审阅用)

使用方式:

```
validator = DataValidator(config)
result = validator.run_all(sales_df, payment_df, inventory_df)
if result['can_proceed']:
    print('数据质量通过, 进入分析')
else:
    print(f"阻断: {result['critical_count']} 个 CRITICAL 项未解决")
```

设计说明:

- 每个 `validate_*` 方法返回独立的 `DataFrame`, 最后由 `run_all` 合并
- 不将 `self.issues` 作为跨方法累积状态, 避免重复计数

"""

```
import pandas as pd
import numpy as np
from datetime import datetime
from pathlib import Path
import json
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class DataValidator:
```

```
    """数据质量验证器
```

```
    每个 validate_* 方法返回 (issues_df, can_proceed) 元组,
    其中 issues_df 是该检查发现的异常明细, can_proceed 表示是否阻断分析。
```

```
    """
```

```
    def __init__(self, config):
        self.config = config
        # self.issues 仅用于 run_all 阶段的汇总, 不在单个 validate 方法中累积
```

```
    def _detect_column_name(self, df, standard_name):
        """根据 COLUMN_MAPPING 自动匹配实际列名
```

```
        设计理由: 源数据字段名不固定, 硬编码列名会导致运行时 KeyError。
```

```
        使用配置的别名映射自动匹配, 找不到时返回 None。
```

```
    """
```

```

aliases = self.config['COLUMN_MAPPING'].get(standard_name, [standard_name])
for alias in aliases:
    if alias in df.columns:
        return alias
return None

def validate_sales_detail(self, df):
    """工作表 1: 销售明细表验证"""
    issues = []

    # 动态解析列名
    col_客户编号 = self._detect_column_name(df, '客户编号')
    col_客户名称 = self._detect_column_name(df, '客户名称')
    col_渠道类型 = self._detect_column_name(df, '渠道类型')
    col_产品品种 = self._detect_column_name(df, '产品品种')
    col_金额 = self._detect_column_name(df, '金额')
    col_成本 = self._detect_column_name(df, '成本')
    col_发货数量 = self._detect_column_name(df, '发货数量')
    col_单价 = self._detect_column_name(df, '单价')
    col_业务负责人 = self._detect_column_name(df, '业务负责人')
    col_订单编号 = self._detect_column_name(df, '订单编号')

    checks = {}

    # 客户编号检查 (CRITICAL)
    if col_客户编号:
        checks['客户编号空值'] = {
            'count': int(df[col_客户编号].isna().sum()),
            'severity': 'CRITICAL',
        }
    if col_客户名称:
        dupes = df.groupby(col_客户编号)[col_客户名称].nunique()
        checks['客户编号重复映射'] = {
            'count': int((dupes > 1).sum()),
            'severity': 'CRITICAL',
        }

    # 渠道类型检查 (CRITICAL)
    if col_渠道类型:
        checks['渠道类型缺失'] = {
            'count': int(df[col_渠道类型].isna().sum()),
            'severity': 'CRITICAL',
        }
        checks['渠道类型非法值'] = {
            'count': int((~df[col_渠道类型].isin(['直销', '代理'])).sum()),
            'severity': 'WARNING',
        }

```



```

# 产品品种检查
if col_产品品种:
    checks['产品品种空值'] = {
        'count': int(df[col_产品品种].isna().sum()),
        'severity': 'WARNING',
    }

# 金额检查
if col_金额:
    checks['金额空值或零'] = {
        'count': int((df[col_金额].isna() | (df[col_金额] == 0)).sum()),
        'severity': 'WARNING',
    }

# 成本检查
if col_成本:
    checks['成本空值'] = {
        'count': int(df[col_成本].isna().sum()),
        'severity': 'WARNING',
    }

# 负数未标注
if col_发货数量 and col_金额:
    checks['负数未标注'] = {
        'count': int(((df[col_发货数量] < 0) | (df[col_金额] < 0)).sum()),
        'severity': 'WARNING',
    }

# 单价金额数量一致性
if all([col_金额, col_发货数量, col_单价]):
    with np.errstate(divide='ignore', invalid='ignore'):
        ratio = df[col_金额] / df[col_发货数量]
        checks['单价金额数量不一致'] = {
            'count': int((np.abs(ratio - df[col_单价]) > 0.01).sum()),
            'severity': 'WARNING',
        }

# 销售负责人含产品文本
if col_业务负责人:
    checks['销售负责人含产品文本'] = {
        'count': int(
            df[col_业务负责人]
            .astype(str)
            .str.contains('芯片|模组|产品|系列', na=False, regex=True)
            .sum()
        ),
        'severity': 'WARNING',
    }

```

```

# 订单编号校验
if col_订单编号:
    checks['订单编号空值'] = {
        'count': int(df[col_订单编号].isna().sum()),
        'severity': 'WARNING',
    }

# 汇总结果
for check_name, info in checks.items():
    if info['count'] > 0:
        issues.append({
            '检查项': check_name,
            '异常数': info['count'],
            '严重级别': info['severity'],
            '建议处理': '阻断分析' if info['severity'] == 'CRITICAL' else '标注后进入',
        })

issues_df = pd.DataFrame(issues) if issues else pd.DataFrame(
    columns=['检查项', '异常数', '严重级别', '建议处理']
)
can_proceed = not any(i['severity'] == 'CRITICAL' for i in checks.values() if i['count'])

return issues_df, can_proceed

def validate_payment(self, sales_df, payment_df):
    """工作表 2: 对账回款表验证 (PDF 方案核心)"""
    issues = []

    col_客户编号 = self._detect_column_name(sales_df, '客户编号')
    col_金额 = self._detect_column_name(sales_df, '金额')
    col_发货日期 = self._detect_column_name(sales_df, '发货日期')

    if not all([col_客户编号, col_金额, col_发货日期]):
        logger.warning('回款验证: 缺少必要字段, 跳过')
        return pd.DataFrame(columns=['检查项', '异常数', '严重级别', '建议处理']), True

# 应收金额与销售明细汇总一致性
try:
    sales_by_month = sales_df.groupby(
        [col_客户编号, pd.Grouper(key=col_发货日期, freq='ME')]
    )[col_金额].sum().reset_index()
    sales_by_month['对账月度'] = sales_by_month[col_发货日期].dt.strftime('%Y-%m')

    if '客户编号' in payment_df.columns and '对账月度' in payment_df.columns:
        merged = payment_df.merge(
            sales_by_month,
            on=['客户编号', '对账月度'],

```

```

        how='outer',
        suffixes=('_回款', '_销售'),
    )

    if '应收金额' in merged.columns and col_金额 in merged.columns:
        mismatch = merged[
            (merged['应收金额'] - merged[col_金额]).abs() > 0.01
        ]
        if len(mismatch) > 0:
            issues.append({
                '检查项': '回款应收与销售汇总不匹配',
                '异常数': len(mismatch),
                '严重级别': 'CRITICAL',
                '建议处理': '阻断分析',
            })
    except Exception as e:
        issues.append({
            '检查项': '回款验证执行异常',
            '异常数': 1,
            '严重级别': 'WARNING',
            '建议处理': f'检查数据格式: {str(e)}',
        })

    issues_df = pd.DataFrame(issues) if issues else pd.DataFrame(
        columns=['检查项', '异常数', '严重级别', '建议处理']
    )
    can_proceed = not any(i['严重级别'] == 'CRITICAL' for i in issues)

    return issues_df, can_proceed

def validate_inventory(self, inventory_df):
    """库存数据验证（文字版新增）"""
    issues = []
    col_产品品种 = self._detect_column_name(inventory_df, '产品品种')

    if not col_产品品种:
        logger.warning('库存验证: 未找到产品品种列, 跳过')
        return pd.DataFrame(columns=['检查项', '异常数', '严重级别', '建议处理']), True

    checks = {
        '产品品种空值': {
            'count': int(inventory_df[col_产品品种].isna().sum()),
            'severity': 'WARNING',
        },
    }

    if '当前库存数量' in inventory_df.columns:
        checks['库存数量为负'] = {

```

```

        'count': int((inventory_df['当前库存数量'] < 0).sum()),
        'severity': 'WARNING',
    }

if '入库日期' in inventory_df.columns:
    future = pd.to_datetime(inventory_df['入库日期'], errors='coerce') > datetime.now()
    checks['入库日期未来'] = {
        'count': int(future.sum()),
        'severity': 'WARNING',
    }

if '标准成本' in inventory_df.columns:
    checks['成本缺失'] = {
        'count': int(inventory_df['标准成本'].isna().sum()),
        'severity': 'WARNING',
    }

for check_name, info in checks.items():
    if info['count'] > 0:
        issues.append({
            '检查项': check_name,
            '异常数': info['count'],
            '严重级别': info['severity'],
            '建议处理': '标注后进入',
        })

issues_df = pd.DataFrame(issues) if issues else pd.DataFrame(
    columns=['检查项', '异常数', '严重级别', '建议处理']
)
return issues_df, True # 库存检查不阻断

def run_all(self, sales_df, payment_df=None, inventory_df=None):
    """执行全部数据质量验证，输出汇总报告

    返回：
        dict: {
            'can_proceed': bool,
            'critical_count': int,
            'warning_count': int,
            'issues': list[dict],
            'report_time': str,
            'etl_run_id': str,
        }
    """

```

设计理由：

- 独立返回每个验证结果，最后汇总，避免状态累积
- can\_proceed 是下游 Pipeline 的开关
- report\_time 使用数据最大日期（若可获取）或当前时间

```

"""
all_issues = []
can_proceed = True

# 执行各验证方法
sales_issues, sales_ok = self.validate_sales_detail(sales_df)
all_issues.append(sales_issues)
can_proceed = can_proceed and sales_ok

if payment_df is not None:
    pmt_issues, pmt_ok = self.validate_payment(sales_df, payment_df)
    all_issues.append(pmt_issues)
    can_proceed = can_proceed and pmt_ok

if inventory_df is not None:
    inv_issues, inv_ok = self.validate_inventory(inventory_df)
    all_issues.append(inv_issues)
    can_proceed = can_proceed and inv_ok

# 合并所有问题
combined = pd.concat(all_issues, ignore_index=True) if all_issues else pd.DataFrame(
    columns=['检查项', '异常数', '严重级别', '建议处理']
)

# 统计
critical_count = int((combined['严重级别'] == 'CRITICAL').sum())
warning_count = int((combined['严重级别'] == 'WARNING').sum())

summary = {
    '验证时间': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
    'etl_run_id': self.config.get('ETL_RUN_ID', 'unknown'),
    '是否可进入分析': bool(can_proceed and critical_count == 0),
    'CRITICAL项数': critical_count,
    'WARNING项数': warning_count,
    '详细记录': combined.to_dict('records') if len(combined) > 0 else [],
}

# 输出 JSON
output_dir = Path(self.config.get('OUTPUT_DIR', 'output'))
output_dir.mkdir(parents=True, exist_ok=True)

with open(output_dir / 'data_validation_report.json', 'w', encoding='utf-8') as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)

# 输出 Excel
if len(combined) > 0:
    combined.to_excel(output_dir / '数据质量验证报告.xlsx', index=False)
else:

```

```
pd.DataFrame({'状态': ['无异常发现']}).to_excel(
    output_dir / '数据质量验证报告.xlsx', index=False
)

return summary
```

## 1.5 数据质量通过标准

| 检查项        | 阻断标准     | 通过标准     | 建议处理                  |
|------------|----------|----------|-----------------------|
| 客户编号空值     | CRITICAL | = 0      | 补全客户编号或删除该行           |
| 客户编号重复映射   | CRITICAL | = 0      | 统一客户编号规则              |
| 渠道类型缺失     | CRITICAL | = 0      | 补齐渠道标记                |
| 回款应收与销售不匹配 | CRITICAL | < 1% 差异行 | 对账后修复                 |
| 单价金额数量不一致  | WARNING  | < 5%     | 标注后进入                 |
| 金额空值或零     | WARNING  | < 5%     | 补全或删除                 |
| 业务负责人含产品文本 | WARNING  | = 0      | 修正姓名列（常见于从 ERP 导出时错位） |

# 第二层：共享数据管道（Silver 层）（第 2-3 周）

设计理由：

- 所有清洗逻辑在 Silver 层一次性完成，Gold 层的分析引擎不需要处理脏数据
- 清洗顺序有严格的依赖关系：工作日调整 → 退货匹配 → 买赠还原 → 项目备货标记 → 毛利率钳制
- 清理日志记录每一步的变更量，用于排查和审计
- 如果源数据无订单编号（ALLOW\_EMPTY\_ORDER\_ID=True），跳过订单级聚合

## 2.1 清洗逻辑融合实现

"""

Silver 层清洗 – 融合两方案的核心清洗规则

输入: Bronze 层原始数据 CSV

输出: 三张 Silver 表 + 清洗日志

清洗顺序（依赖关系，不可调换）：

1. `apply_workday_adjustment` – 调整数量/金额，后续所有分析基于调整值
2. `match_returns` – 退货冲销，需要原始发货日期窗口
3. `allocate_promotion` – 买赠还原，需基于净数量净金额
4. `flag_project_stock` – 项目备货标记，需基于净数量
5. `apply_winsorization` – 毛利率钳制，需基于净金额净成本

新人须知：

- 不要改变 `run()` 中清洗函数的调用顺序
- 如果新增清洗规则，仔细评估应该放在哪个步骤之前/之后
- 调试时请查看 `silver/cleaning_log.json` 了解每一步的变更量

"""

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from pathlib import Path
import json
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class SilverTransformer:
```

```
    def __init__(self, config):
        self.config = config
        self.cleaning_log = []
        self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')
```

```
    # -----
    # 列名自动映射工具
    # -----
```

```
    def _col(self, df, standard_name):
        """根据 COLUMN_MAPPING 返回实际列名，找不到则返回 None"""
        aliases = self.config['COLUMN_MAPPING'].get(standard_name, [standard_name])
        for alias in aliases:
            if alias in df.columns:
                return alias
        return None
```

```
def load_bronze(self, file_path: str) -> pd.DataFrame:
    """加载 Bronze 层数据，保留原始形态

    设计理由: Bronze 是原始快照，不在此处做任何清洗。
    即使数据有误，也应保留现场供排查。
    """

    df = pd.read_csv(file_path)
    # 尝试解析日期列
    date_cols = [self._col(df, '发货日期'), self._col(df, '下单日期')]
    for col in date_cols:
        if col:
            df[col] = pd.to_datetime(df[col], errors='coerce')

    self.cleaning_log.append(f"加载 Bronze 数据: {len(df)} 行, {len(df.columns)} 列")
    return df
```

```
# -----
# 清洗步骤 1: 工作日调整
# -----
```

```
def apply_workday_adjustment(self, df: pd.DataFrame) -> pd.DataFrame:
    """
```

文字版核心增强: 工作日调整

设计理由:

- 2 月 (~20 工作日) 和 3 月 (~23 工作日) 的自然销量差异可达 15%
- 不调整会导致 2 月「异常下滑」的误判
- 调整后值仅用于趋势分析，财务报告仍使用原始值

计算公式:

调整数量 = 原始数量 ÷ (当月实际工作日 / 21)

调整金额 = 原始金额 ÷ (当月实际工作日 / 21)

RFC:

- 标记工作日 < 标准 70% 的月份（如春节所在月）为无效月
- 这些月份的环比数据应被排除在趋势判断之外

```
"""
```

```
col_发货日期 = self._col(df, '发货日期')
```

```
col_发货数量 = self._col(df, '发货数量')
```

```
col_金额 = self._col(df, '金额')
```

```
if not all([col_发货日期, col_发货数量, col_金额]):
    logger.warning('工作日调整: 缺少必要字段, 跳过')
    df['调整数量'] = df[col_发货数量] if col_发货数量 else np.nan
    df['调整金额'] = df[col_金额] if col_金额 else np.nan
    df['有效工作月'] = True
    return df
```



```

# 加载节假日
holidays_file = self.config['CLEANING_RULES']['workday_adjustment']['holidays_file']
try:
    holidays_df = pd.read_excel(holidays_file)
    holiday_dates = set(pd.to_datetime(holidays_df.iloc[:, 0]))
except (FileNotFoundError, Exception) as e:
    logger.warning(f"节假日文件未找到 ({holidays_file}), 使用默认日历: {e}")
    holiday_dates = set()

df['发货年月'] = df[col_发货日期].dt.to_period('M')

def count_workdays(year_month):
    """计算给定月份的实际工作日（排除周末和节假日）"""
    start = year_month.to_timestamp()
    end = year_month.to_timestamp(how='end')
    days = pd.date_range(start, end, freq='D')
    workdays = [d for d in days if d.weekday() < 5 and d not in holiday_dates]
    return len(workdays)

# 计算每个月的实际工作日
unique_months = df['发货年月'].unique()
month_workdays = {m: count_workdays(m) for m in unique_months}

df['当月实际工作日'] = df['发货年月'].map(month_workdays)
standard = self.config['CLEANING_RULES']['workday_adjustment']['standard_workdays_per_mor']
df['工作日系数'] = df['当月实际工作日'] / standard

# 计算调整值
df['调整数量'] = df[col_发货数量] / df['工作日系数'].replace(0, np.nan)
df['调整金额'] = df[col_金额] / df['工作日系数'].replace(0, np.nan)

# 标记无效月份（工作日不足标准 70%）
min_ratio = self.config['CLEANING_RULES']['workday_adjustment']['min_valid_workday_ratio']
df['有效工作月'] = df['工作日系数'] >= min_ratio

invalid_count = int((~df['有效工作月']).sum())
self.cleaning_log.append(
    f"工作日调整完成: 共 {len(unique_months)} 个月, "
    f"标记 {invalid_count} 行为无效工作月"
)
return df

# -----
# 清洗步骤 2: 退货冲销（后进先出匹配）
# -----
def match_returns(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    文字版核心: 退货冲销（后进先出匹配）
    """

```

设计理由：

- 退货记录以负数发货数量标记，需要匹配到原始正向销售
- 采用后进先出（LIFO）原则：先匹配最近的销售
- 匹配窗口 90 天（可配置），超过 90 天的退货无法匹配
- 匹配后的净数量 = 正向数量 - 匹配的退货数量

边界情况：

- 无正向记录的退货 → 保留为负值，标记为「未匹配退货」
- 同一笔销售可能被多次退货部分匹配 → 跟踪未匹配数量

.....

```
col_客户编号 = self._col(df, '客户编号')
col_产品品种 = self._col(df, '产品品种')
col_发货日期 = self._col(df, '发货日期')
col_发货数量 = self._col(df, '发货数量')
col_金额 = self._col(df, '金额')
col_成本 = self._col(df, '成本')

# 设置净值的初始值（等于原始值）
df['净数量'] = df[col_发货数量]
df['净金额'] = df[col_金额]
df['净成本'] = df.get(col_成本, pd.Series(0, index=df.index))
df['已匹配'] = False

# 分离退货和正向交易
mask_negative = df[col_发货数量] < 0
returns = df[mask_negative].copy()
normals = df[~mask_negative].copy()
normals['未匹配数量'] = normals[col_发货数量].values # 可被退货匹配的数量

if len(returns) == 0:
    self.cleaning_log.append("退货匹配：无退货记录")
    return df

match_window = self.config['CLEANING_RULES']['return_matching']['match_window_days']
matched_pairs = [] # 记录（退货索引，匹配数量，匹配金额，匹配成本）

for idx, ret in returns.iterrows():
    # 在窗口中寻找正向记录（后进先出：按日期降序）
    window_start = ret[col_发货日期] - timedelta(days=match_window)
    candidates = normals[
        (normals[col_客户编号] == ret[col_客户编号]) &
        (normals[col_产品品种] == ret[col_产品品种]) &
        (normals[col_发货日期] >= window_start) &
        (normals[col_发货日期] <= ret[col_发货日期]) &
        (normals['未匹配数量'] > 0)
    ].sort_values(col_发货日期, ascending=False)
```

```

remaining_qty = abs(ret[col_发货数量])
matched_qty = 0
matched_amt = 0
matched_cost = 0

for cand_idx in candidates.index:
    if remaining_qty <= 0:
        break
    cand = normals.loc[cand_idx]
    match_qty = min(remaining_qty, cand['未匹配数量'])
    ratio = match_qty / cand[col_发货数量] if cand[col_发货数量] != 0 else 0

    matched_qty += match_qty
    matched_amt += cand[col_金额] * ratio
    matched_cost += cand[col_成本] * ratio if col_成本 else 0

    # 减少正向记录的未匹配数量
    normals.loc[cand_idx, '未匹配数量'] -= match_qty
    remaining_qty -= match_qty

if matched_qty > 0:
    matched_pairs.append({
        '退货索引': idx,
        '匹配数量': matched_qty,
        '匹配金额': matched_amt,
        '匹配成本': matched_cost,
        '未完全匹配': remaining_qty > 0,
    })

# 更新原始 DataFrame 的净数量
for mp in matched_pairs:
    idx = mp['退货索引']
    # 部分匹配: 净值 = 原退货值 + 匹配回的正向值
    df.loc[idx, '净数量'] = -mp['匹配数量']
    df.loc[idx, '净金额'] = -mp['匹配金额']
    df.loc[idx, '净成本'] = -mp['匹配成本']
    df.loc[idx, '已匹配'] = True

total_returns = len(returns)
matched_count = sum(1 for mp in matched_pairs)
partial_count = sum(1 for mp in matched_pairs if mp['未完全匹配'])

self.cleaning_log.append(
    f"退货匹配完成: {total_returns} 笔退货, "
    f"{matched_count} 笔部分或完全匹配, "
    f"{partial_count} 笔未完全匹配"
)
return df

```

```

# -----
# 清洗步骤 3: 买赠还原
# -----
def allocate_promotion(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    文字版核心: 买赠还原

    设计理由:
    - 「买十送一」等促销导致部分产品行金额为 0 或偏低
    - 同一次促销活动的特征是: 同一业务员 + 同一客户/代理商 + 同一日期
    - 按数量比例分摊实收金额, 还原每个产品的等效净单价

    算法:
    1. 用「业务员 + 代理商 + 发货日期」生成促销组标识
    2. 组内总实收金额按各产品数量比例分摊
    3. 如果等效净单价 < 标准单价 × 50%, 标记为买赠异常单
    """
    col_业务员 = self._col(df, '业务员')
    col_代理商名称 = self._col(df, '代理商名称')
    col_发货日期 = self._col(df, '发货日期')

    if not col_业务员:
        logger.warning('买赠还原: 缺少业务员字段, 跳过')
        df['等效净单价'] = df[self._col(df, '金额')] / df[self._col(df, '发货数量')]
        df['买赠标记'] = False
        return df

    # 生成促销组标识
    date_str = df[col_发货日期].dt.strftime('%Y%m%d') if col_发货日期 else ''
    agent_col = col_代理商名称 if col_代理商名称 else 'unknown'
    df['促销组标识'] = (
        df[col_业务员].astype(str) + '_' +
        df[agent_col].astype(str) + '_' +
        date_str
    )

    def _allocate_group(group):
        """组内按数量比例分摊"""
        total_net_amt = group['净金额'].sum()
        total_qty = group['净数量'].sum()
        if total_qty == 0 or total_net_amt == 0:
            group['等效净单价'] = group['净金额'] / group['净数量'].replace(0, np.nan)
            return group

    # 等效净单价 = 总实收 × (单产品数量占比) / 单产品数量 = 总实收 / 总数量
    group['等效净单价'] = total_net_amt / total_qty
    return group

```

```

df = df.groupby('促销组标识', group_keys=False).apply(_allocate_group)

# 标记买赠异常（等效净单价显著低于应有单价）
col_单价 = self._col(df, '单价')
if col_单价:
    df['买赠标记'] = df['等效净单价'] < df[col_单价] * 0.5
else:
    df['买赠标记'] = False

flagged = int(df['买赠标记'].sum())
self.cleaning_log.append(f"买赠还原完成：{flagged} 笔标记为买赠")
return df

# -----
# 清洗步骤 4：项目备货标记
# -----
def flag_project_stock(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    文字版核心：项目备货标记

    设计理由：
    - 大客户项目备货表现为单次采购量远超历史平均水平
    - 用「前 12 月月均量 × 3」作为阈值
    - 标记后的记录不纳入常规采购行为分析（如采购间隔计算）

    修改说明（相对原方案）：
    - 原方案用 Period 对象做 merge key，改为统一的字符串年月字段
    - 保证类型兼容和可读性
    """
    col_客户编号 = self._col(df, '客户编号')
    col_产品品种 = self._col(df, '产品品种')
    col_发货日期 = self._col(df, '发货日期')

    if not all([col_客户编号, col_产品品种]):
        df['项目备货标记'] = False
        return df

# 计算每个客户-产品组合的月度净数量
df['发货年月'] = df[col_发货日期].dt.to_period('M').astype(str)
monthly_qty = df.groupby([
    col_客户编号, col_产品品种, '发货年月'
])[col_净数量].sum().reset_index()

# 前 12 月月均量（用 rolling 窗口，排除当月）
monthly_qty = monthly_qty.sort_values([col_客户编号, col_产品品种, '发货年月'])
monthly_qty['近12月月均量'] = monthly_qty.groupby(
    [col_客户编号, col_产品品种]

```

```

) ['净数量'].transform(
    lambda x: x.rolling(12, min_periods=3).mean().shift(1)
)

# 统一用字符串年月做 merge
df = df.merge(
    monthly_qty[['col_客户编号', col_产品品种, '发货年月', '近12月月均量']],
    on=[col_客户编号, col_产品品种, '发货年月'],
    how='left'
)

threshold = self.config['CLEANING_RULES']['project_stock_flag']['threshold_multiplier']
df['项目备货标记'] = (
    df['净数量'] > df['近12月月均量'] * threshold
).fillna(False)

flagged = int(df['项目备货标记'].sum())
self.cleaning_log.append(f"项目备货标记: {flagged} 笔")
return df

# -----
# 清洗步骤 5: 毛利率 Winsorization
# -----
def apply_winsorization(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    PDF 方案核心: 毛利率 Winsorization (极端值钳制)

    设计理由:
    - 毛利率低于 -50% 通常是数据错误 (成本 > 售价 × 1.5)
    - 毛利率高于 75% 通常是买赠或特价清库导致的暂时性异常
    - Winsorization (钳制而非删除) 保留了记录数量, 不影响聚合计数
    - 钳制后的值仅用于比率分析, 实际财务计算仍使用原始值
    """
    if '净金额' not in df.columns or '净成本' not in df.columns:
        logger.warning("毛利率钳制: 缺少净金额/净成本, 跳过")
        return df

    with np.errstate(divide='ignore', invalid='ignore'):
        df['毛利率'] = (df['净金额'] - df['净成本']) / df['净金额']

    cfg = self.config['CLEANING_RULES']['margin_winsorization']
    lower = cfg['lower_clip']
    upper = cfg['upper_clip']

    outlier_count = int(((df['毛利率'] < lower) | (df['毛利率'] > upper)).sum())
    df['毛利率_钳制'] = df['毛利率'].clip(lower, upper)

    self.cleaning_log.append(

```

```

        f"毛利率 Winsorization: {outlier_count} 笔异常值被钳制到 [{lower}, {upper}]"
    )
    return df

# -----
# Silver 表生成
# -----

def _resolve_order_id_col(self, df):
    """解析订单编号列，如果不存在且不允许空则警告"""
    col = self._col(df, '订单编号')
    if col is None and not self.config.get('ALLOW_EMPTY_ORDER_ID', False):
        logger.warning("数据中无订单编号列，订单级分析将被跳过")
    return col

def generate_silver_tables(self, df: pd.DataFrame) -> dict:
    """
    输出三张 Silver 表（PDF 方案标准）

    silver_customer_monthly:
        - 每客户 × 每月一行
        - 用于客户势能分析、RFM 评分

    silver_product_monthly:
        - 每产品 × 每月一行
        - 用于产品生命周期、库存需求估算

    silver_customer_x_product:
        - 每客户 × 每产品 × 每月一行
        - 桥接表，用于交叉分析和购物篮

    所有表均包含 etl_run_id 和 created_at 字段用于审计
    """
    now_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    col_客户编号 = self._col(df, '客户编号')
    col_产品品种 = self._col(df, '产品品种')
    col_发货日期 = self._col(df, '发货日期')

    # 1. silver_customer_monthly
    # 设计理由（ADR-004）：月度金额/数量默认使用工作日调整值，
    # 原始值保留在 原始金额/原始数量 中供对照。
    # 趋势分析使用调整值，财务报告使用原始值。
    agg_dict = {
        '净金额': 'sum',          # → 原始金额（未调整）
        '净成本': 'sum',          # → 月度成本（不调整成本）
        '净数量': 'sum',          # → 原始数量（未调整）
        '调整金额': 'sum',        # → 月度金额（工作日调整后）
        '调整数量': 'sum',        # → 月度数量（工作日调整后）
        col_产品品种: 'nunique',   # → 活跃品种数
    }

```

```

}
order_col = self._resolve_order_id_col(df)
if order_col:
    agg_dict[order_col] = 'nunique'

customer_monthly = df.groupby([
    col_客户编号, df[col_发货日期].dt.to_period('M').astype(str)
]).agg(agg_dict).reset_index()

# 列映射：调整值为主，原始值为参考
base_cols = ['客户编号', '月份', '原始金额', '月度成本', '原始数量',
             '月度金额', '月度数量', '活跃品种数']
if order_col:
    base_cols.append('订单数')
customer_monthly.columns = base_cols[:len(customer_monthly.columns)]

customer_monthly['月度毛利率'] = (
    (customer_monthly['月度金额'] - customer_monthly['月度成本']) /
    customer_monthly['月度金额'].replace(0, np.nan)
)
customer_monthly['原始毛利率'] = (
    (customer_monthly['原始金额'] - customer_monthly['月度成本']) /
    customer_monthly['原始金额'].replace(0, np.nan)
)
customer_monthly['etl_run_id'] = self.etl_run_id
customer_monthly['created_at'] = now_str

# 2. silver_product_monthly
product_monthly = df.groupby([
    col_产品品种, df[col_发货日期].dt.to_period('M').astype(str)
]).agg({
    '净金额': 'sum',      # → 原始金额
    '净成本': 'sum',      # → 月度成本
    '净数量': 'sum',      # → 原始数量
    '调整金额': 'sum',    # → 月度金额
    '调整数量': 'sum',    # → 月度数量
    col_客户编号: 'nunique',
}).reset_index()
product_monthly.columns = ['产品品种', '月份', '原始金额', '月度成本', '原始数量',
                          '月度金额', '月度数量', '活跃客户数']
product_monthly['etl_run_id'] = self.etl_run_id
product_monthly['created_at'] = now_str

# 3. silver_customer_x_product
cxp_agg = {
    '净金额': 'sum',      # → 原始金额
    '净成本': 'sum',      # → 月度成本
    '净数量': 'sum',      # → 原始数量

```



```

        '调整金额': 'sum',          # → 月度金额
        '调整数量': 'sum',          # → 月度数量
        '等效净单价': 'mean',
        '项目备货标记': 'any',
        '买赠标记': 'any',
    }

    customer_x_product = df.groupby([
        col_客户编号, col_产品品种, df[col_发货日期].dt.to_period('M').astype(str)
    ]).agg(cxp_agg).reset_index()
    customer_x_product.columns = [
        '客户编号', '产品品种', '月份',
        '原始金额', '月度成本', '原始数量',
        '月度金额', '月度数量',
        '等效净单价', '有项目备货', '有买赠',
    ]
    customer_x_product['etl_run_id'] = self.etl_run_id
    customer_x_product['created_at'] = now_str

    return {
        'silver_customer_monthly': customer_monthly,
        'silver_product_monthly': product_monthly,
        'silver_customer_x_product': customer_x_product,
    }

```

```

# -----
# 主入口
# -----

```

```

def run(self, bronze_file: str) -> dict:
    """执行完整 Silver 层转换

```

清洗顺序（有严格依赖关系）：

1. 工作日调整 （生成调整数量/金额）
2. 退货匹配 （设置净数量/净金额/净成本）
3. 买赠还原 （基于净值计算等效单价）
4. 项目备货 （需要净数量+时间窗口）
5. 毛利率钳制 （基于净金额/净成本）

返回: dict[str, pd.DataFrame] – 三张 Silver 表  
 """

```

df = self.load_bronze(bronze_file)

```

# 按依赖顺序执行清洗

```

df = self.apply_workday_adjustment(df)
df = self.match_returns(df)
df = self.allocate_promotion(df)
df = self.flag_project_stock(df)
df = self.apply_winsorization(df)

```

```
silver_tables = self.generate_silver_tables(df)

# 保存清洗日志
silver_dir = Path(self.config.get('SILVER_DIR', 'silver'))
silver_dir.mkdir(parents=True, exist_ok=True)

with open(silver_dir / 'cleaning_log.json', 'w', encoding='utf-8') as f:
    json.dump({
        'etl_run_id': self.etl_run_id,
        'steps': self.cleaning_log,
    }, f, ensure_ascii=False, indent=2)

# 保存 Silver 表
for name, table in silver_tables.items():
    table.to_csv(silver_dir / f'{name}.csv', index=False)

logger.info(f"Silver 层完成: {len(df)} 行 → 3 张聚合表")
return silver_tables
```

2.2 Silver 表输出规范

| 表名                        | 粒度           | 关键字段                                            | 用途            |
|---------------------------|--------------|-------------------------------------------------|---------------|
| silver_customer_monthly   | 客户 × 月份      | 月度金额/数量（工作日调整值）、原始金额/数量（未调整）、月度成本、活跃品种数、订单数、毛利率 | 客户势能、RFM 评分   |
| silver_product_monthly    | 产品 × 月份      | 月度金额/数量（调整值）、原始金额/数量、月度成本、活跃客户数                 | 产品生命周期、库存需求估算 |
| silver_customer_x_product | 客户 × 产品 × 月份 | 月度金额/数量（调整值）、原始金额/数量、月度成本、等效净单价、标记              | 交叉分析、购物篮桥接    |

第三层：核心分析引擎（Gold 层）（第 3-5 周）

3.1 客户分析引擎

- 设计理由：
- 10 个维度覆盖客户经营势能、毛利拆解、价格治理、品种生命周期、回款信用、机会/风险评分

- 每个维度独立计算方法，最后在 `run()` 中汇总
- 评分体系使用百分位排名而非绝对值，消除量纲影响
- 参考日期固定（ `config.ANALYSIS_REFERENCE_DATE` ），保证结果可复现
- 所有输出表包含 `etl_run_id` 和 `created_at` 审计字段

"""

客户分析核心引擎 - 10 个维度计算 + 评分

输入: Silver 层表

输出:

- gold\_customer\_profile (每客户一行, 60+ 指标)
- gold\_customer\_x\_product (每客户 × 每产品 × 月份)

新人须知:

- 每个 calc\_dimension\_\* 方法返回一个以客户编号为 key 的 DataFrame
- run() 方法将所有维度 merge 在一起后计算评分
- 如果需要新增维度, 创建一个 calc\_dimension\_xx 方法并在 run() 中调用

"""

```
import pandas as pd
import numpy as np
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from datetime import datetime, timedelta
import logging

logger = logging.getLogger(__name__)

class CustomerAnalysisEngine:
    def __init__(self, config, silver_tables):
        self.config = config
        self.silver = silver_tables
        self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')

    def _get_reference_date(self, df):
        """获取分析参考日期

        设计理由:
        - 使用固定参考日期而非 pd.to_datetime('now')
        - 保证每次运行结果可复现
        - 参考日期来源优先级:
            1. config 中明确指定的日期
            2. 数据中的最大月份
        """
        ref = self.config.get('ANALYSIS_REFERENCE_DATE', 'auto')
        if ref != 'auto':
            return pd.to_datetime(ref)

        # 从数据中取最大月份
        if '月份' in df.columns:
```

```

        return pd.to_datetime(df['月份']).max()
    return pd.to_datetime('2026-05-01') # fallback

# -----
# 经营势能维度
# -----
def calc_dimension_01_business_momentum(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    维度 01: 客户经营势能

    指标: 月度收入、毛利额、毛利率、加权 ASP、发货数量
    对比: 环比(上月)、同比(去年同月)
    趋势: 连续增长/下滑月数

    计算方式:
    - 近 12 月聚合(求和、均值、标准差)
    - 加权 ASP = 近 12 月总收入 / 近 12 月总数量
    - 连续趋势用逐月 diff 的正负判断
    """
    ref_date = self._get_reference_date(df)
    latest_month = ref_date
    window_start = latest_month - pd.DateOffset(months=12)

    # 确保月份是日期类型
    df = df.copy()
    df['月份'] = pd.to_datetime(df['月份'])

    recent = df[(df['月份'] >= window_start) & (df['月份'] <= latest_month)].copy()

    if len(recent) == 0:
        logger.warning("维度 01: 12 月窗口内无数据")
        return pd.DataFrame({'客户编号': df['客户编号'].unique()})

    metrics = recent.groupby('客户编号').agg(
        近12月收入=('月度金额', 'sum'),
        月均收入=('月度金额', 'mean'),
        收入波动=('月度金额', 'std'),
        近12月成本=('月度成本', 'sum'),
        近12月数量=('月度数量', 'sum'),
        活跃月数=('月份', 'count'),
    ).reset_index()

    metrics['近12月毛利'] = metrics['近12月收入'] - metrics['近12月成本']
    metrics['近12月毛利率'] = metrics['近12月毛利'] / metrics['近12月收入'].replace(0, np.nan)
    metrics['加权ASP'] = metrics['近12月收入'] / metrics['近12月数量'].replace(0, np.nan)

    # 环比同比
    latest = df[df['月份'] == latest_month].set_index('客户编号')['月度金额']

```

```

prev_month = df[df['月份'] == latest_month - pd.DateOffset(months=1)].set_index('客户编号')
yoy_month = df[(df['月份'] >= latest_month - pd.DateOffset(months=12)) &
                (df['月份'] < latest_month - pd.DateOffset(months=11))]

if not yoy_month.empty:
    yoy_month = yoy_month.groupby('客户编号')['月度金额'].sum()

metrics['收入环比'] = metrics['客户编号'].map(
    lambda cid: (latest.get(cid, np.nan) - prev_month.get(cid, np.nan)) /
                prev_month.get(cid, np.nan) if prev_month.get(cid, 0) != 0 else np.nan
)

if not yoy_month.empty:
    metrics['收入同比'] = metrics['客户编号'].map(
        lambda cid: (latest.get(cid, np.nan) - yoy_month.get(cid, np.nan)) /
                    yoy_month.get(cid, np.nan) if yoy_month.get(cid, 0) != 0 else np.nan
    )
else:
    metrics['收入同比'] = np.nan

# 连续增长/下滑月数
def _consecutive_trend(series):
    """计算最大连续增长和下滑月数"""
    if len(series) < 2:
        return 0, 0
    diffs = series.diff().dropna()
    growth_streak = 0
    decline_streak = 0
    max_growth = 0
    max_decline = 0

    for d in diffs:
        if d > 0:
            growth_streak += 1
            decline_streak = 0
            max_growth = max(max_growth, growth_streak)
        elif d < 0:
            decline_streak += 1
            growth_streak = 0
            max_decline = max(max_decline, decline_streak)
        else:
            growth_streak = 0
            decline_streak = 0

    return max_growth, max_decline

trends = recent.sort_values('月份').groupby('客户编号')['月度金额'].apply(_consecutive_trend)
metrics['连续增长月数'] = metrics['客户编号'].map(lambda cid: trends.get(cid, (0, 0))[0])

```

```

metrics['连续下滑月数'] = metrics['客户编号'].map(lambda cid: trends.get(cid, (0, 0))[1])

return metrics

# -----
# 毛利三因子拆解
# -----
def calc_dimension_02_margin_decomposition(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    维度 02: 毛利三因子拆解

    公式:
    
$$\Delta \text{毛利} \approx \Delta \text{数量} \times \text{期初单位毛利} + \Delta \text{销售单价} \times \text{期末数量} - \Delta \text{单位成本} \times \text{期末数量}$$


    设计理由:
    - 当客户毛利下降时, 需要回答「是卖少了、卖便宜了、还是成本涨了?」
    - 三因子拆解比简单看毛利率变化更有诊断价值
    """
    exp = self.silver['silver_customer_x_product'].copy()
    exp['月份'] = pd.to_datetime(exp['月份'])
    exp = exp.sort_values(['客户编号', '产品品种', '月份'])

    # 计算单位值
    exp['单位毛利'] = (exp['月度金额'] - exp['月度成本']) / exp['月度数量'].replace(0, np.nan)
    exp['单价'] = exp['月度金额'] / exp['月度数量'].replace(0, np.nan)
    exp['单位成本'] = exp['月度成本'] / exp['月度数量'].replace(0, np.nan)

    # 上期值
    exp['上期数量'] = exp.groupby(['客户编号', '产品品种'])['月度数量'].shift(1)
    exp['上期单价'] = exp.groupby(['客户编号', '产品品种'])['单价'].shift(1)
    exp['上期单位成本'] = exp.groupby(['客户编号', '产品品种'])['单位成本'].shift(1)
    exp['上期单位毛利'] = exp.groupby(['客户编号', '产品品种'])['单位毛利'].shift(1)

    # 三因子
    exp['数量因子'] = (exp['月度数量'] - exp['上期数量']) * exp['上期单位毛利']
    exp['价格因子'] = (exp['单价'] - exp['上期单价']) * exp['月度数量']
    exp['成本因子'] = -(exp['单位成本'] - exp['上期单位成本']) * exp['月度数量']

    customer_decomp = exp.groupby('客户编号').agg({
        '数量因子': 'sum',
        '价格因子': 'sum',
        '成本因子': 'sum',
    }).reset_index()

    customer_decomp['毛利变化'] = (
        customer_decomp['数量因子'] +
        customer_decomp['价格因子'] +
        customer_decomp['成本因子']
    )

```

```

)

return customer_decomp

# -----
# 价格治理维度
# -----
def calc_dimension_03_pricing_governance(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    维度 03: 价格治理

    指标:
    - 整体 ASP (按客户汇总)
    - 品种价格偏离度 (客户 x 品种 ASP 与全体同品种中位 ASP 的差异)
    - 低价品种收入占比
    """
    cxp = self.silver['silver_customer_x_product'].copy()
    cxp['品种ASP'] = cxp['月度金额'] / cxp['月度数量'].replace(0, np.nan)

    # 客户整体 ASP
    customer_asp = cxp.groupby('客户编号').apply(
        lambda x: x['月度金额'].sum() / x['月度数量'].sum() if x['月度数量'].sum() > 0 else np.nan,
        include_groups=False
    ).reset_index(name='整体ASP')

    # 品种中位 ASP
    median_by_product = cxp.groupby('产品品种')['品种ASP'].median()
    cxp['中位ASP'] = cxp['产品品种'].map(median_by_product)
    cxp['价格偏离率'] = (cxp['品种ASP'] - cxp['中位ASP']) / cxp['中位ASP'].replace(0, np.nan)

    # 价格带
    asp_quantiles = cxp['品种ASP'].quantile([0.25, 0.5, 0.75])

    def _price_band(asp):
        if pd.isna(asp):
            return '未知'
        if asp <= asp_quantiles[0.25]:
            return '低价带'
        elif asp <= asp_quantiles[0.75]:
            return '中价带'
        else:
            return '高价带'

    cxp['价格带'] = cxp['品种ASP'].apply(_price_band)

    # 低价品种收入占比
    customer_low_price = cxp.groupby('客户编号').apply(
        lambda x: x[x['价格带'] == '低价带']['月度金额'].sum() / x['月度金额'].sum()

```



```

        if x['月度金额'].sum() > 0 else 0,
        include_groups=False
    ).reset_index(name='低价品种收入占比')

    return customer_asp.merge(customer_low_price, on='客户编号', how='left')

```

```

# -----

```

```

# 品种生命周期维度

```

```

# -----

```

```

def calc_dimension_05_sku_lifecycle(self, df: pd.DataFrame) -> tuple:

```

```

    """

```

维度 05：品种生命周期状态机

状态：

- 新增：该品种在该客户首次出现
- 持续：上月和当月均有发货
- 消失：上月有、当月无发货
- 复活：消失至少 1 个月后再次出现
- 放量：当月数量较上月增长超过 100%
- 衰退：连续 3 个月数量下降且当月下降超过 30%

修正说明（相对原方案）：

- 衰退判定改用三层 shift 检查环比递减，替代 rolling window
- 原方案的 `rolling(3).apply(lambda x: all(x < x.iloc[0] * 0.7))` 语义有误
- 新逻辑：连续 3 个月逐月下降，且当月 < 上月 × 0.7

```

    """

```

```

    cxp = self.silver['silver_customer_x_product'].copy()

```

```

    cxp['月份'] = pd.to_datetime(cxp['月份'])

```

```

    cxp = cxp.sort_values(['客户编号', '产品品种', '月份'])

```

# 首次出现月份

```

    first_month = cxp.groupby(['客户编号', '产品品种'])['月份'].min()

```

```

    cxp['首次出现'] = cxp.set_index(['客户编号', '产品品种']).index.map(first_month)

```

```

    cxp['是否新增'] = cxp['月份'] == cxp['首次出现']

```

# 偏移量

```

    cxp['上月数量'] = cxp.groupby(['客户编号', '产品品种'])['月度数量'].shift(1)

```

```

    cxp['前月数量'] = cxp.groupby(['客户编号', '产品品种'])['月度数量'].shift(2)

```

```

    cxp['上月有发货'] = cxp['上月数量'].notna() & (cxp['上月数量'] > 0)

```

# 状态判断（按优先级）

```

    conditions = [

```

```

        cxp['是否新增'],

```

```

        cxp['月度数量'] > cxp['上月数量'] * 2, # 放量（先于持续判断）

```

```

        ((cxp['月度数量'] < cxp['上月数量'] * 0.7) & # 衰退

```

```

         (cxp['上月数量'] < cxp['前月数量']) & # 连续 3 月下降

```

```

         cxp['前月数量'].notna()),

```

```

        cxp['上月有发货'] & (cxp['月度数量'] > 0), # 持续

```

```

        cxp['上月有发货'] & (cxp['月度数量'] == 0), # 消失
        (~cxp['上月有发货']) & (cxp['月度数量'] > 0), # 复活
    ]
    choices = ['新增', '放量', '衰退', '持续', '消失', '复活']

    cxp['状态'] = np.select(conditions, choices, default='持续')

    # 集中度: Top 3 品种收入占比
    customer_concentration = cxp.groupby('客户编号').apply(
        lambda x: x.groupby('产品品种')['月度金额'].sum().nlargest(3).sum() /
            x['月度金额'].sum() if x['月度金额'].sum() > 0 else 0,
        include_groups=False
    ).reset_index(name='Top3品种收入占比')

    return cxp, customer_concentration

# -----
# 客户采购健康度（文字版补充）
# -----
def calc_text_version_health_metrics(self, df: pd.DataFrame) -> pd.DataFrame:
    """
    文字版补充: 客户采购健康度

    指标:
    - 常规平均采购间隔（剔除新品和项目备货的前 6 个月）
    - 距上次采购天数（基于参考日期）
    - 采购中断预警（距上次采购 > 常规间隔 × 1.5）
    - Top 5 产品集中度
    - 品类接受度（主导品类占比）

    修正说明（相对原方案）:
    - 参考日期使用 config 的 ANALYSIS_REFERENCE_DATE
    - 采购间隔计算使用正确的 datetime diff
    """
    ref_date = self._get_reference_date(df)
    cxp = self.silver['silver_customer_x_product'].copy()
    cxp['月份'] = pd.to_datetime(cxp['月份'])

    # 排除新品期和项目备货
    min_month = cxp.groupby('客户编号')['月份'].transform('min')
    regular = cxp[
        (~cxp.get('有项目备货', pd.Series(False, index=cxp.index))) &
        (cxp['月份'] >= min_month + pd.DateOffset(months=6)) # 排除前 6 个月
    ].copy()

    # 常规平均采购间隔
    def _avg_interval(group):
        sorted_months = group['月份'].sort_values()

```

```

    if len(sorted_months) < 2:
        return np.nan
    intervals = sorted_months.diff().dt.days.iloc[1:]
    return intervals.mean()

intervals = regular.groupby('客户编号', group_keys=False).apply(
    _avg_interval, include_groups=False
).reset_index(name='常规平均采购间隔')

# 距上次采购天数（基于参考日期）
last_purchase = exp.groupby('客户编号')['月份'].max()
days_since = (ref_date - last_purchase).dt.days

intervals['距上次采购天数'] = intervals['客户编号'].map(days_since)
intervals['采购中断预警'] = (
    intervals['距上次采购天数'] > intervals['常规平均采购间隔'] * 1.5
)

# Top 5 产品集中度
concentration = exp.groupby('客户编号').apply(
    lambda x: x.groupby('产品品种')['月度金额'].sum().nlargest(5).sum() /
        x['月度金额'].sum() if x['月度金额'].sum() > 0 else 0,
    include_groups=False
).reset_index(name='Top5产品集中度')
concentration['强依赖标记'] = concentration['Top5产品集中度'] > 0.7

# 品类接受度
if '产品线' in exp.columns:
    category_share = exp.groupby(['客户编号', '产品线'])['月度金额'].sum()
    customer_total = exp.groupby('客户编号')['月度金额'].sum()
    share = category_share / customer_total
    dominant = share.groupby('客户编号').max().reset_index()
    dominant.columns = ['客户编号', '主导品类占比']
    dominant['品类机会'] = np.where(
        dominant['主导品类占比'] > 0.6,
        '优先导入同品类新品',
        '观察未打开品类'
    )
else:
    dominant = pd.DataFrame({
        '客户编号': exp['客户编号'].unique(),
        '品类机会': '数据缺失（需产品线信息）',
    })

result = intervals.merge(concentration, on='客户编号', how='left')
result = result.merge(dominant, on='客户编号', how='left')
return result

```

```

# -----
# 机会/风险评分
# -----
def calc_dimension_09_opportunity_risk_score(self, metrics_df: pd.DataFrame) -> pd.DataFrame:
    """
    维度 09: 机会/风险评分（顶层输出）

    机会评分（百分位数排名，每项 0-100）：
    - 收入规模（25%）
    - 增长趋势（20%）
    - 毛利质量（20%）
    - 新品渗透率（15%）—— 如不可获取则使用默认值
    - 产品线广度（10%）
    - 回款健康度（10%）

    风险评分：
    - 连续下滑月数（25%）
    - 品种集中度（15%）
    - 回款逾期（10%）
    - 关键品种衰退占比（10%）
    - 剩余因子分摊剩余权重

    等级划分：四分位数
    """
    # 机会评分
    opportunity = pd.Series(0.0, index=metrics_df.index)

    # 收入规模（25%）
    if '近12月收入' in metrics_df.columns:
        opportunity += metrics_df['近12月收入'].rank(pct=True) * 25

    # 增长趋势（20%）
    if '收入同比' in metrics_df.columns and '收入环比' in metrics_df.columns:
        trend = (metrics_df['收入同比'].fillna(0) + metrics_df['收入环比'].fillna(0)) / 2
        opportunity += trend.rank(pct=True) * 20

    # 毛利质量（20%）
    if '近12月毛利率' in metrics_df.columns:
        margin_quality = metrics_df['近12月毛利率'].fillna(0)
        # 结合毛利率水平和稳定性
        if '收入波动' in metrics_df.columns:
            vol_norm = metrics_df['收入波动'].fillna(0)
            vol_norm = vol_norm / vol_norm.max() if vol_norm.max() > 0 else 0
            margin_quality = margin_quality * (1 - vol_norm)
        opportunity += margin_quality.rank(pct=True) * 20

    # 新品渗透率（15%）—— 使用默认值 0.5 如果不可计算
    new_product_rate = metrics_df.get('新品渗透率', pd.Series(0.5, index=metrics_df.index))

```

```

opportunity += new_product_rate.rank(pct=True) * 15

# 产品线广度 (10%)
if '活跃品种数' in metrics_df.columns:
    opportunity += metrics_df['活跃品种数'].rank(pct=True) * 10
else:
    opportunity += 5 # 默认中等

# 回款健康度 (10%)
if '信用等级' in metrics_df.columns:
    credit_score = metrics_df['信用等级'].map({'A': 1.0, 'B': 0.7, 'C': 0.4, 'D': 0.1}).fillna(0)
    opportunity += credit_score * 10
else:
    opportunity += 5

metrics_df['机会分'] = opportunity / 100

# 风险评分
risk = pd.Series(0.0, index=metrics_df.index)

# 连续下滑 (25%)
if '连续下滑月数' in metrics_df.columns:
    risk += metrics_df['连续下滑月数'].clip(0, 6) / 6 * 25

# 品种集中度 (15%)
if 'Top3品种收入占比' in metrics_df.columns:
    risk += metrics_df['Top3品种收入占比'].fillna(0) * 15

# 回款逾期 (10%)
if '逾期率' in metrics_df.columns:
    risk += metrics_df['逾期率'].fillna(0) * 10

# 关键品种衰退 (10%)
if '衰退品种占比' in metrics_df.columns:
    risk += metrics_df['衰退品种占比'].fillna(0) * 10
# 注: 衰退品种占比需要在 calc_dimension_05 中补充计算
# 当前版本: 如果缺失则不计算此项 (剩余权重分配给连续下滑)

# ASP 跌幅 (20%)
risk_asp = metrics_df.get('ASP同比变化', pd.Series(0, index=metrics_df.index))
risk += risk_asp.clip(-1, 0).abs() * 20

# 毛利率跌幅 (20%)
risk_margin = metrics_df.get('毛利率同比变化', pd.Series(0, index=metrics_df.index))
risk += risk_margin.clip(-1, 0).abs() * 20

metrics_df['风险分'] = risk / 100

```

```

# 等级划分
try:
    metrics_df['机会等级'] = pd.qcut(
        metrics_df['机会分'].rank(method='first'), q=4,
        labels=['D', 'C', 'B', 'A']
    )
except ValueError:
    metrics_df['机会等级'] = 'C'

try:
    metrics_df['风险等级'] = pd.qcut(
        metrics_df['风险分'].rank(method='first'), q=4,
        labels=['低', '中', '高', '极高']
    )
except ValueError:
    metrics_df['风险等级'] = '中'

return metrics_df

# -----
# 动作建议生成
# -----
def _generate_actions(self, row):
    """根据评分生成三条建议

    设计理由:
    - 建议 1: 基于流失/机会等级的最紧急事项
    - 建议 2: 基于集中度/价格偏离的交叉销售或价格审查
    - 建议 3: 基于品类机会的新品导入
    """
    actions = ['', '', '']

    # 建议 1: 紧急度最高
    if row.get('风险等级') == '极高':
        decline_months = row.get('连续下滑月数', 0)
        actions[0] = f"紧急回访: 该客户连续下滑 {decline_months} 月, 需评估流失风险"
    elif row.get('机会等级') == 'A':
        actions[0] = "重点跟进: 高机会客户, 建议推动新品渗透"
    elif row.get('采购中断预警', False):
        days_since = row.get('距上次采购天数', 0)
        actions[0] = f"采购中断预警: 距上次采购 {days_since} 天, 超常规间隔 1.5 倍"
    else:
        actions[0] = "维持常规跟进"

    # 建议 2: 价格/集中度
    if row.get('强依赖标记', False):
        actions[1] = "交叉销售: 前 5 产品占比 > 70%, 推动品类扩展"
    else:

```

```

        # 如果产品组合中有衰退品，补充交叉销售建议
        actions[1] = "持续关注产品组合变化"

    # 建议 3：品类导入
    if row.get('品类机会') == '优先导入同品类新品':
        actions[2] = "新品导入：该客户某品类占比 > 60%，是同品类新品最佳切入点"

    return actions

# -----
# 主入口
# -----
def run(self) -> pd.DataFrame:
    """执行完整客户分析

    返回: gold_customer_profile DataFrame
    """
    now_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    df = self.silver['silver_customer_monthly']
    df['月份'] = pd.to_datetime(df['月份'])

    # 依次计算各维度
    logger.info("计算维度 01: 经营势能")
    dim01 = self.calc_dimension_01_business_momentum(df)

    logger.info("计算维度 02: 毛利三因子")
    dim02 = self.calc_dimension_02_margin_decomposition(df)

    logger.info("计算维度 03: 价格治理")
    dim03 = self.calc_dimension_03_pricing_governance(df)

    logger.info("计算维度 05: 品种生命周期")
    _, dim05_conc = self.calc_dimension_05_sku_lifecycle(df)

    logger.info("计算客户健康度")
    health = self.calc_text_version_health_metrics(df)

    # 合并所有维度
    profile = dim01.merge(dim02, on='客户编号', how='left')
    profile = profile.merge(dim03, on='客户编号', how='left')
    profile = profile.merge(dim05_conc, on='客户编号', how='left')
    profile = profile.merge(health, on='客户编号', how='left')

    # 计算评分
    profile = self.calc_dimension_09_opportunity_risk_score(profile)

    # 生成动作建议
    actions = profile.apply(self._generate_actions, axis=1, result_type='expand')

```

```

profile['建议动作1'] = actions.iloc[:, 0]
profile['建议动作2'] = actions.iloc[:, 1]
profile['建议动作3'] = actions.iloc[:, 2]

# 审计字段
profile['etl_run_id'] = self.etl_run_id
profile['created_at'] = now_str

logger.info(f"客户分析完成: {len(profile)} 条客户画像")
return profile

```

## ## 3.2 库存分析引擎

> \*\*设计理由\*\*:

- > - 库存数据的核心价值在于分类和策略建议，而非简单的数量统计
- > - 四分类（畅销/呆滞可激活/呆滞风险/成长储备）对应不同的处置策略
- > - 所有库存依赖的销售侧字段（如近6月有发货、近6月波动系数等）在此引擎中显式计算
- > - 修正说明（相对原方案）：原方案的分类条件引用了多个从未计算的字段，现已全部补全

```python

"""

库存分析引擎 - 文字版第四章

输入：库存数据 + Silver 层销售数据

输出：gold\_inventory\_product

策略映射：

- 畅销品：周转快 + 多客户 → 保障供应
- 呆滞可激活型：周转慢 + 曾有客户 → 阶梯折扣清库
- 呆滞风险型：周转极慢 + 无客户 → 根因分析
- 成长储备型：快速增长 + 库存不足 → 动态安全库存
- 单一客户高波动：依赖度高 → 备货风险提示

新人须知：

- 引擎需要先计算派生字段（calc\_derived\_fields），再执行分类
- 派生字段的计算依赖 Silver 层的产品月度表和客户-产品表

"""

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import logging

```

```

logger = logging.getLogger(__name__)

```

```

class InventoryAnalysisEngine:

```



```
def __init__(self, config, inventory_df, silver_tables):
    self.config = config
    self.inventory = inventory_df
    self.silver = silver_tables
    self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')

def calc_derived_fields(self, inv: pd.DataFrame) -> pd.DataFrame:
```

```
"""
```

计算库存分类所需的全部派生字段

设计理由:

- 原方案引用了近6月有发货、近12月有发货、有历史客户、近3月销量环比、库存覆盖天数、近6月波动系数等字段，但从未计算
- 现在统一在此处从 Silver 表派生出

计算步骤:

1. 近3月调整日均销量
2. 库存周转天数 = 当前库存 / 日均销量
3. 库存覆盖天数 = 当前库存 / 日均销量（同周转天数，但单位不同）
4. 近6月/近12月是否有发货
5. 有历史客户
6. 近3月销量环比
7. 近6月波动系数（变异系数 =  $\text{std}/\text{mean}$ ）

```
"""
```

```
prod_monthly = self.silver['silver_product_monthly'].copy()
prod_monthly['月份'] = pd.to_datetime(prod_monthly['月份'])
latest_month = prod_monthly['月份'].max()

# 1. 近3月调整日均销量
window_3m = prod_monthly[prod_monthly['月份'] >= latest_month - pd.DateOffset(months=3)]
daily_sales_3m = window_3m.groupby('产品品种')['月度数量'].sum() / 90.0
inv['近3月调整日均销量'] = inv['产品品种'].map(daily_sales_3m).fillna(0)

# 2. 库存周转天数 / 库存覆盖天数
inv['库存周转天数'] = np.where(
    inv['近3月调整日均销量'] > 0,
    inv['当前库存数量'] / inv['近3月调整日均销量'],
    999 # 无销售时设为极大值
)
inv['库存覆盖天数'] = inv['库存周转天数']

# 3. 近6月/近12月是否有发货
for months, col_name in [(6, '近6月有发货'), (12, '近12月有发货')]:
    window = prod_monthly[prod_monthly['月份'] >= latest_month - pd.DateOffset(months=months)]
    has_shipment = window.groupby('产品品种')['月度数量'].sum() > 0
    inv[col_name] = inv['产品品种'].map(has_shipment).fillna(False)

# 4. 有历史客户（Silver 层中任何出现过的产品）
```

```

exp = self.silver['silver_customer_x_product']
has_customers = exp.groupby('产品品种')['客户编号'].nunique() > 0
inv['有历史客户'] = inv['产品品种'].map(has_customers).fillna(False)

# 5. 活跃客户数（近3月有交易的客户数）
window_3m_cxp = exp.copy() if '月份' not in exp.columns else exp
if '月份' in window_3m_cxp.columns:
    window_3m_cxp['月份'] = pd.to_datetime(window_3m_cxp['月份'])
    window_3m_cxp = window_3m_cxp[
        window_3m_cxp['月份'] >= latest_month - pd.DateOffset(months=3)
    ]
active_customers = window_3m_cxp.groupby('产品品种')['客户编号'].nunique()
inv['活跃客户数'] = inv['产品品种'].map(active_customers).fillna(0).astype(int)

# 6. 近3月销量环比（本月 vs 上月）
product_monthly_sorted = prod_monthly.sort_values(['产品品种', '月份'])
product_monthly_sorted['上月数量'] = product_monthly_sorted.groupby('产品品种')['月度数量'].
product_monthly_sorted['环比'] = (
    (product_monthly_sorted['月度数量'] - product_monthly_sorted['上月数量']) /
    product_monthly_sorted['上月数量'].replace(0, np.nan)
)
latest_2m = product_monthly_sorted[
    product_monthly_sorted['月份'] >= latest_month - pd.DateOffset(months=1)
]
latest_环比 = latest_2m.groupby('产品品种')['环比'].mean()
inv['近3月销量环比'] = inv['产品品种'].map(latest_环比).fillna(0)

# 7. 近6月波动系数（变异系数）
window_6m = prod_monthly[prod_monthly['月份'] >= latest_month - pd.DateOffset(months=6)]
volatility = window_6m.groupby('产品品种').agg(
    mean_qty=('月度数量', 'mean'),
    std_qty=('月度数量', 'std'),
)
volatility['波动系数'] = volatility['std_qty'] / volatility['mean_qty'].replace(0, np.nan)
inv['近6月波动系数'] = inv['产品品种'].map(volatility['波动系数']).fillna(0)

# 8. 产品线
if '产品线' in prod_monthly.columns:
    product_line = prod_monthly.groupby('产品品种')['产品线'].first()
    inv['产品线'] = inv['产品品种'].map(product_line)

return inv

def classify_inventory(self, inv: pd.DataFrame) -> pd.DataFrame:
    """
    库存四分类 + 单一客户高波动标记

    分类优先级（互斥，先满足者胜出）：

```

1. 畅销品 - 周转  $\leq 30$  天 + 活跃客户  $\geq 3$
2. 成长储备型 - 近3月环比  $\geq 20\%$  + 活跃客户  $\geq 2$  + 覆盖  $\leq 14$  天
3. 呆滞可激活型 - 周转  $\geq 90$  天 + 近6月有发货 + 有历史客户
4. 呆滞风险型 - 周转  $\geq 180$  天 + 近12月无发货 + 无历史客户

单一客户高波动不互斥，作为独立标记

"""

```
cfg = self.config['INVENTORY_CLASSIFICATION']
```

```
# 按优先级逐层分类
```

```
inv['库存分类'] = '其他'
```

```
# 畅销品
```

```
mask_fast = (  
    (inv['库存周转天数'] <= cfg['fast_moving']['max_turnover_days']) &  
    (inv['活跃客户数'] >= cfg['fast_moving']['min_active_customers'])  
)  
inv.loc[mask_fast, '库存分类'] = '畅销品'
```

```
# 成长储备型
```

```
mask_growth = (  
    (inv['近3月销量环比'] >= cfg['growth_reserve']['min_growth_3m']) &  
    (inv['活跃客户数'] >= cfg['growth_reserve']['min_active_customers']) &  
    (inv['库存覆盖天数'] <= cfg['growth_reserve']['max_stock_coverage_days'])  
)  
inv.loc[mask_growth, '库存分类'] = '成长储备型'
```

```
# 呆滞可激活型
```

```
mask_dormant_act = (  
    (inv['库存周转天数'] >= cfg['dormant_activatable']['min_turnover_days']) &  
    (inv['近6月有发货']) &  
    (inv['有历史客户'])  
)  
inv.loc[mask_dormant_act, '库存分类'] = '呆滞可激活型'
```

```
# 呆滞风险型
```

```
mask_dormant_risk = (  
    (inv['库存周转天数'] >= cfg['dormant_risk']['min_turnover_days']) &  
    (~inv['近12月有发货']) &  
    (~inv['有历史客户'])  
)  
inv.loc[mask_dormant_risk, '库存分类'] = '呆滞风险型'
```

```
# 单一客户高波动（独立标记，不与分类互斥）
```

```
mask_single_vol = (  
    (inv['活跃客户数'] == 1) &  
    (inv['近6月波动系数'] > cfg['single_customer_high_volatility']['condition'].split('>'))  
)
```

```

inv['备货风险标记'] = mask_single_vol

# 策略建议
strategy_map = {
    '畅销品': cfg['fast_moving']['strategy'],
    '呆滞可激活型': cfg['dormant_activatable']['strategy'],
    '呆滞风险型': cfg['dormant_risk']['strategy'],
    '成长储备型': cfg['growth_reserve']['strategy'],
    '其他': '常规监控',
}
inv['建议策略'] = inv['库存分类'].map(strategy_map)

return inv

def run(self) -> pd.DataFrame:
    """执行完整库存分析"""
    now_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    inv = self.inventory.copy()
    inv = self.calc_derived_fields(inv)
    inv = self.classify_inventory(inv)

    # 审计字段
    inv['etl_run_id'] = self.etl_run_id
    inv['created_at'] = now_str

    logger.info(f"库存分析完成: {len(inv)} 个产品, "
                f"畅销品 {(inv['库存分类']=='畅销品').sum()}, "
                f"呆滞风险 {(inv['库存分类']=='呆滞风险型').sum()}")

    return inv

```

### ## 3.3 定价建议引擎 {#33-定价建议引擎 }

- > \*\*设计理由\*\*:
- > - 定价建议分为两个版本: 销售员可见版 (不显示成本/毛利率) 和总监完整版
- > - 降价策略的核心是盈亏平衡计算: 降价多少可以保本?
- > - 弹性系数不再硬编码为 **-1.5**, 改为按产品品种从历史数据中拟合
- > - 当数据不足以拟合时, 使用保守默认值 **-1.0**

```

```python
"""

```

定价建议引擎 - 文字版第六章

输入: Silver 层数据 + 成本数据

输出:

- gold\_pricing\_recommendation (销售员可见版本 + 总监完整版)

策略类型:

1. 提价建议: 价格长期低于同类客户均价且毛利微薄
2. 以价换量: 产品成熟/衰退期, 降价换取可盈利的增量

新人须知:

- generate\_sales\_rep\_view 输出的列是销售员在 BI 中能看到的內容
- full\_df (总监版本) 包含成本和毛利率, 需要权限保护

"""

```
import pandas as pd
import numpy as np
from datetime import datetime
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class PricingRecommendationEngine:
    def __init__(self, config, silver_tables, cost_data):
        self.config = config
        self.silver = silver_tables
        self.cost_data = cost_data
        self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')

    def _fit_elasticity(self, csp: pd.DataFrame) -> pd.Series:
        """
        按产品品种拟合价格弹性系数
```

设计理由:

- 原方案弹性系数硬编码为 -1.5, 不同品类弹性差异极大
- 工业品弹性约 -0.3 ~ -0.8, 消费品约 -1.0 ~ -2.5
- 此处用对数线性回归拟合:  $\ln(\text{数量}) \sim \ln(\text{价格})$
- 至少需要 3 次价格变动才能得到可靠估计

返回: Series, index=产品品种, value=弹性系数

"""

```
cfg = self.config['PRICING']['markdown_strategy']['elasticity']
default = cfg['default']
clip_min, clip_max = cfg['clip_range']
min_obs = cfg['min_observations']
```

```
def _fit_group(group):
    """对单个产品的价格-数量序列拟合弹性"""
    prices = group['等效净单价'].dropna()
    qtys = group['月度数量'].dropna()
    if len(prices) < min_obs or len(qtys) < min_obs:
        return default
    # 使用共同的 index
```

```

common = prices.index.intersection(qtys.index)
if len(common) < min_obs:
    return default

log_p = np.log(prices[common].values)
log_q = np.log(qtys[common].values)

# 过滤无效值
valid = np.isfinite(log_p) & np.isfinite(log_q)
if valid.sum() < min_obs:
    return default

try:
    slope = np.polyfit(log_p[valid], log_q[valid], 1)[0]
    return float(np.clip(slope, clip_min, clip_max))
except np.linalg.LinAlgError:
    return default

elasticity = cxp.groupby('产品品种', group_keys=False).apply(
    _fit_group, include_groups=False
)
return elasticity

def calc_price_uplift_opportunities(self) -> pd.DataFrame:
    """
    常规报价建议：提价空间识别

    逻辑：如果某客户-产品的等效净单价持续低于同类客户均价，
    且毛利率低于公司平均水平，建议提升价格到合理区间。
    """
    cxp = self.silver['silver_customer_x_product'].copy()

    # 同类客户均价（产品 median）
    product_median = cxp.groupby('产品品种')['等效净单价'].median()
    product_p75 = cxp.groupby('产品品种')['等效净单价'].quantile(0.75)

    # 客户-产品级别
    customer_product_price = cxp.groupby(['客户编号', '产品品种']).agg(
        等效净单价=('等效净单价', 'mean'),
        月度金额=('月度金额', 'sum'),
        月度数量=('月度数量', 'sum'),
    ).reset_index()

    customer_product_price['同类客户均价'] = customer_product_price['产品品种'].map(product_median)
    customer_product_price['同类客户P75'] = customer_product_price['产品品种'].map(product_p75)

    # 价格差距（相对于同类客户均价）
    customer_product_price['价格差距'] = (

```

```

        customer_product_price['同类客户均价'] - customer_product_price['等效净单价']
    ) / customer_product_price['同类客户均价'].replace(0, np.nan)

# 合并成本（用于毛利率计算 - 仅总监可见）
if self.cost_data is not None and '产品品种' in self.cost_data.columns:
    cost_col = '标准成本' if '标准成本' in self.cost_data.columns else self.cost_data.columns[0]
    customer_product_price = customer_product_price.merge(
        self.cost_data[['产品品种', cost_col]],
        on='产品品种',
        how='left',
    )
    customer_product_price['标准成本'] = customer_product_price[cost_col]
    customer_product_price['毛利率'] = (
        customer_product_price['等效净单价'] - customer_product_price['标准成本']
    ) / customer_product_price['等效净单价'].replace(0, np.nan)
else:
    customer_product_price['标准成本'] = np.nan
    customer_product_price['毛利率'] = np.nan

# 提价建议
company_avg_margin = customer_product_price['毛利率'].mean()
customer_product_price['提价建议'] = (
    (customer_product_price['价格差距'] > 0.10) &
    (customer_product_price['毛利率'] < company_avg_margin)
).fillna(False)

# 建议价格区间
customer_product_price['建议报价下限'] = customer_product_price['同类客户均价'] * 0.95
customer_product_price['建议报价上限'] = customer_product_price['同类客户P75']

# 预计年度采购额变化
customer_product_price['预计年度采购额变化'] = (
    customer_product_price['月度数量'] * 12 *
    (customer_product_price['建议报价上限'] - customer_product_price['等效净单价'])
)

# 合并业务负责人（用于权限过滤）
if '业务负责人' in cxp.columns:
    sales_rep_map = cxp.groupby(['客户编号', '产品品种'])['业务负责人'].first()
    customer_product_price['业务负责人'] = customer_product_price.set_index(
        ['客户编号', '产品品种']
    ).index.map(sales_rep_map)

return customer_product_price

def calc_markdown_strategy(self, eligible_df: pd.DataFrame) -> pd.DataFrame:
    """
    降价增量策略

```

设计理由：

- 降价的目标不是「薄利多销」，而是「总毛利额增长」
- 约束：降价后目标销量  $\geq$  盈亏平衡销量  $\times 1.2$ （容错余量）
- 约束：降价后总毛利额  $>$  降价前总毛利额  $\times 1.1$ （至少增长 10%）
- 弹性系数从数据中拟合，非硬编码

输出：

- 总监版本（全部字段）
- 销售员版本（隐藏成本/毛利率/盈亏平衡点）

修正说明（相对原方案）：

- 弹性系数改为按产品品种从数据中拟合
- 销售员版本增加了建议价格和目标销量字段

"""

```
cfg = self.config['PRICING']['markdown_strategy']

# 拟合弹性系数
cxp = self.silver['silver_customer_x_product'].copy()
elasticity_series = self._fit_elasticity(cxp)

# 合并弹性系数
eligible_df['弹性系数E'] = eligible_df['产品品种'].map(elasticity_series).fillna(
    cfg['elasticity']['default']
)

# 盈亏平衡销量
eligible_df['盈亏平衡销量'] = (
    (eligible_df['等效净单价'] - eligible_df['标准成本'].fillna(0)) *
    eligible_df['月度数量']
) / (eligible_df['建议报价下限'] - eligible_df['标准成本'].fillna(0) + 1e-10)

# 基于弹性预估目标销量
price_change_ratio = (
    eligible_df['建议报价下限'] - eligible_df['等效净单价']
) / eligible_df['等效净单价'].replace(0, 1e-10)

eligible_df['目标销量'] = eligible_df['月度数量'] * (
    1 + eligible_df['弹性系数E'] * price_change_ratio /
    (1 - price_change_ratio + 1e-10)
)

# 约束检查
eligible_df['满足约束'] = (
    (eligible_df['目标销量'] >= 1.2 * eligible_df['盈亏平衡销量']) &
    (
        (eligible_df['建议报价下限'] - eligible_df['标准成本'].fillna(0)) *
        eligible_df['目标销量'] >
```



```

        (eligible_df['等效净单价'] - eligible_df['标准成本'].fillna(0)) *
        eligible_df['月度数量'] * 1.10
    )
).fillna(False)

# 构建销售员可见版本
sales_rep = eligible_df[eligible_df['满足约束']].copy()
sales_rep['策略类型'] = '以价换量'
sales_rep['展示_当前售价'] = sales_rep['等效净单价']
sales_rep['展示_建议价格'] = sales_rep['建议报价下限']
sales_rep['展示_目标销量'] = sales_rep['目标销量']
sales_rep['展示_预计采购额变化'] = (
    sales_rep['展示_目标销量'] * sales_rep['展示_建议价格'] -
    sales_rep['月度数量'] * sales_rep['展示_当前售价']
)
sales_rep['展示_销量底线'] = 1.2 * sales_rep['盈亏平衡销量']

# 隐藏财务字段（替换为占位符）
hidden = ['标准成本', '毛利率', '盈亏平衡销量', '弹性系数E']
for col in hidden:
    if col in sales_rep.columns:
        sales_rep[col] = '[权限不足]'

return sales_rep

def generate_sales_rep_view(self, markdown_df: pd.DataFrame) -> pd.DataFrame:
    """
    生成销售员可见的定价建议（去财务化）

    输出列：
    - 业务负责人、客户编号、产品品种（标识）
    - 历史成交均价、建议价格（价格信息）
    - 预计年度采购额变化、策略类型（行动信息）
    - 销量底线（约束条件）
    """
    visible_cols = [
        '业务负责人', '客户编号', '产品品种',
        '展示_当前售价', '同类客户均价', '建议报价下限', '建议报价上限',
        '展示_建议价格', '展示_目标销量',
        '展示_预计采购额变化', '策略类型', '展示_销量底线',
    ]

    # 只选择存在的列
    available = [c for c in visible_cols if c in markdown_df.columns]
    if not available:
        logger.warning("定价建议：销售员视图无可用例")
        return pd.DataFrame()

```

```

result = markdown_df[available].copy()
result = result.rename(columns={
    '展示_当前售价': '历史成交均价',
    '展示_建议价格': '建议本次报价',
    '展示_目标销量': '争取销量目标',
    '展示_预计采购额变化': '预计年度采购额变化',
    '展示_销量底线': '销量底线',
})

return result

def generate_director_view(self, full_df: pd.DataFrame, markdown_df: pd.DataFrame) -> pd.DataFrame:
    """
    生成总监完整版本（含成本和毛利率）

    合并提价建议和降价策略：
    - 提价建议行 + 降价策略行（策略类型=提价/以价换量）
    - 所有财务字段可见
    """
    # 提价建议
    uplift = full_df[full_df['提价建议']].copy()
    if len(uplift) > 0:
        uplift['策略类型'] = '常规提价'
        uplift['建议报价'] = uplift['建议报价上限']

    # 降价策略
    markdown = markdown_df.copy()
    # 恢复隐藏字段
    for col in ['标准成本', '毛利率', '盈亏平衡销量', '弹性系数E']:
        if col in markdown.columns:
            markdown[col] = full_df.loc[markdown.index, col].values

    combined = pd.concat([uplift, markdown], ignore_index=True) if len(uplift) > 0 else markdown
    return combined

def run(self) -> tuple:
    """执行完整定价分析

    返回: (sales_rep_view, director_view)
    - sales_rep_view: 销售员可见，无成本/毛利率
    - director_view: 完整数据，含成本/毛利率
    """
    now_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    full = self.calc_price_uplift_opportunities()
    markdown = self.calc_markdown_strategy(full)

    sales_rep = self.generate_sales_rep_view(markdown)

```

```

director = self.generate_director_view(full, markdown)

# 审计字段
for df in [sales_rep, director]:
    if df is not None and len(df) > 0:
        df['etl_run_id'] = self.etl_run_id
        df['created_at'] = now_str

logger.info(f"定价建议完成: {len(sales_rep)} 条销售员建议, "
            f"{len(director)} 条总监建议")
return sales_rep, director

```

# 第四层：购物篮分析（高级模块，可选）（第 5-6 周）

- > \*\*设计理由\*\*:
- > - 购物篮分析是纯增量的增值模块，不依赖它主流程也能运行
- > - 三种篮子（订单/时间/项目）覆盖不同的购买关联场景
- > - 综合评分体系加权多个指标，避免单一指标偏差
- >
- > 修正说明（相对原方案）:
- > - 原方案 `build_baskets` 中 `cxp['月份']` 是字符串，尝试 `.dt` 访问器会 `AttributeError`
- > - 项目篮子用月度粒度  $\pm 30$  天语义模糊，改为在客户粒度上取产品首次出现前后 60 天
- > - 时间篮子改为按年月字符串分组，避免日期类型问题

```
```python
```

```
````
```

购物篮分析 - 文字版 3.4 节

使用 `mlxtend` 实现关联规则

篮子定义:

1. 订单篮子（50%）：同一订单号内的产品组合
2. 时间篮子（30%）：同一客户同一自然月的购买组合
3. 项目篮子（20%）：某产品首次出现前后 60 天内的其他购买

前置条件:

- 数据中必须有订单编号（否则跳过订单篮子）
- 当 `ALLOW_EMPTY_ORDER_ID = True` 时，订单篮子权重重新分配给时间篮子

```
````
```

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import logging

```

```
logger = logging.getLogger(__name__)
```

```
class MarketBasketAnalyzer:
```

```
    def __init__(self, config, silver_tables):
        self.config = config
        self.silver = silver_tables
        self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')
```

```
    def build_baskets(self) -> pd.DataFrame:
```

```
        """
```

```
        构建三种购物篮
```

```
        设计理由:
```

- 三种篮子捕捉不同层级的关联:
- 订单: 同一笔采购的强关联 (如打印机+墨盒)
- 时间: 月度采购习惯 (如每月固定采购的办公用品组合)
- 项目: 产品导入前后的配套需求

```
        修正说明 (相对原方案):
```

- 修复: `cxp['月份']` 可能是字符串, 强制转为 `datetime`
- 修复: 项目篮子不再用月度  $\pm 30$  天 (粒度太大), 改用产品首次出现前 60 天到后 60 天 (在客户维度上)
- 时间篮子: 用年月字符串分组, 避免 `Period` 类型问题

```
        """
```

```
        cxp = self.silver['silver_customer_x_product'].copy()
```

```
        # 确保月份是 datetime
```

```
        cxp['月份'] = pd.to_datetime(cxp['月份'])
```

```
        baskets = []
```

```
        order_col = self._resolve_order_id_col(cxp)
```

```
        # 1. 订单篮子 (50%)
```

```
        if order_col:
```

```
            order_baskets = cxp.dropna(subset=[order_col]).groupby(order_col)['产品品种'].apply(lambda x: x[x.apply(len) >= 2]) # 至少 2 个产品
            order_baskets = order_baskets[order_baskets.apply(len) >= 2]
            order_df = order_baskets.reset_index()
            order_df.columns = ['篮子ID', '产品品种']
            order_df['篮子类型'] = 'order'
            order_df['权重'] = 0.50
            baskets.append(order_df)
            logger.info(f"订单篮子: {len(order_df)} 个篮子")
```

```
        else:
```

```
            logger.warning("无订单编号, 订单篮子跳过")
```

```
        # 2. 时间篮子 (30%, 若无订单编号则提升到 50%)
```

```
        time_weight = 0.30 if order_col else 0.50
```

```
        cxp['年月'] = cxp['月份'].dt.strftime('%Y-%m') # 字符串, 避免 Period 类型问题
```

```

time_baskets = exp.groupby(['客户编号', '年月'])['产品品种'].apply(list)
time_baskets = time_baskets[time_baskets.apply(len) >= 2]
time_df = time_baskets.reset_index()
time_df.columns = ['篮子ID', '年月', '产品品种']
time_df['篮子类型'] = 'time'
time_df['权重'] = time_weight
baskets.append(time_df)

```

# 3. 项目篮子（20%，若无订单编号则提升到 30%）

```
project_weight = 0.20 if order_col else 0.30
```

```
def _get_project_baskets(group):
```

```
    """找出客户粒度上的产品首次出现时间
```

对客户内每个产品，找出首次出现的年份月，然后取该月前后 60 天内该客户购买的所有产品。

```
    """
```

```
    first_purchase = group.groupby('产品品种')['月份'].min()
    results = []
```

```

    for product, first_date in first_purchase.items():
        window_start = first_date - timedelta(days=60)
        window_end = first_date + timedelta(days=60)
        window_products = group[
            (group['月份'] >= window_start) &
            (group['月份'] <= window_end) &
            (group['产品品种'] != product)
        ]['产品品种'].unique()

        if len(window_products) >= 2:
            results.append({
                'anchor_product': product,
                'related_products': list(window_products),
            })
    return results

```

```

project_data = exp.groupby('客户编号', group_keys=False).apply(
    _get_project_baskets, include_groups=False
)

```

```

project_rows = []
for items in project_data:
    if items:
        for item in items:
            project_rows.append(item)

```

```

if project_rows:
    project_df = pd.DataFrame(project_rows)

```

```

project_df['篮子类型'] = 'project'
project_df['权重'] = project_weight
# 展平为每行一个产品篮子
expanded = []
for _, row in project_df.iterrows():
    expanded.append({
        '篮子ID': f"project_{row['anchor_product']}",
        '产品品种': row['related_products'],
        '篮子类型': 'project',
        '权重': project_weight,
    })
baskets.append(pd.DataFrame(expanded))
logger.info(f"项目篮子: {len(expanded)} 个篮子")

if not baskets:
    logger.warning("购物篮: 没有足够的交易数据构建篮子")
    return pd.DataFrame()

return pd.concat(baskets, ignore_index=True)

def _resolve_order_id_col(self, df):
    """解析订单编号列"""
    aliases = self.config['COLUMN_MAPPING'].get('订单编号', ['订单编号', 'OrderID'])
    if self.config.get('ALLOW_EMPTY_ORDER_ID', False):
        return None
    for alias in aliases:
        if alias in df.columns:
            return alias
    return None

def mine_association_rules(self, baskets: pd.DataFrame) -> pd.DataFrame:
    """
    挖掘关联规则 (apriori 算法 + lift 过滤)
    """
    if len(baskets) == 0:
        return pd.DataFrame()

    cfg = self.config['MARKET_BASKET']['association_rules']

    # 转换为 one-hot 编码
    te = TransactionEncoder()
    te_ary = te.fit(baskets['产品品种'].tolist()).transform(baskets['产品品种'].tolist())
    df = pd.DataFrame(te_ary, columns=te.columns_)

    # 频繁项集
    frequent_itemsets = apriori(
        df,
        min_support=cfg['min_support'],

```

```

        use_colnames=True,
        low_memory=True,
    )

    if len(frequent_itemsets) == 0:
        logger.warning("购物篮：未发现频繁项集")
        return pd.DataFrame()

# 关联规则
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=cfg['min_confidence'],
)

# 过滤提升度
rules = rules[rules['lift'] > cfg['min_lift']]

return rules

def score_complementary_pairs(self, rules: pd.DataFrame) -> pd.DataFrame:
    """
    综合评分

    评分维度（加权求和）：
    - 加权置信度（30%）
    - 提升度（20%）
    - 首购触发比例（20%）
    - 增量触发比例（15%）
    - 全体覆盖率（10%）
    - KA/AA 覆盖率（5%）
    """
    if len(rules) == 0:
        return rules

    weights = self.config['MARKET_BASKET']['scoring_weights']

    rules['置信度得分'] = rules['confidence'] * 100

    lift_range = rules['lift'].max() - rules['lift'].min()
    if lift_range > 0:
        rules['提升度得分'] = (rules['lift'] - rules['lift'].min()) / lift_range * 100
    else:
        rules['提升度得分'] = 50

# 以下三项简化处理（完整实现需要更复杂的时序交叉验证）
rules['首购触发得分'] = 50 # 应基于时序数据计算
rules['增量触发得分'] = 50 # 应基于时序数据计算

```

```

rules['覆盖率得分'] = 50      # 应从 baskets 计算
rules['KA覆盖率得分'] = 50    # 应基于客户分层计算

rules['综合评分'] = (
    rules['置信度得分'] * weights['weighted_confidence'] +
    rules['提升度得分'] * weights['lift'] +
    rules['首购触发得分'] * weights['first_purchase_trigger'] +
    rules['增量触发得分'] * weights['incremental_trigger'] +
    rules['覆盖率得分'] * weights['overall_coverage'] +
    rules['KA覆盖率得分'] * weights['ka_aa_coverage']
)

rules['互补品对'] = rules['综合评分'] > self.config['MARKET_BASKET']['complementary_thresho

return rules

def generate_recommendations(self, rules: pd.DataFrame, cxp: pd.DataFrame) -> pd.DataFrame:
    """
    生成交叉销售推荐

    对每个互补品对 (X → Y):
        - 找出买了 X 但没买 Y 的客户列表
        - 按目标客户数量排序
    """
    if len(rules) == 0:
        return pd.DataFrame()

    complementary = rules[rules['互补品对']].copy()
    if len(complementary) == 0:
        logger.info("购物篮: 未发现互补品对")
        return pd.DataFrame()

    recommendations = []

    for _, rule in complementary.iterrows():
        antecedents = list(rule['antecedents'])
        consequents = list(rule['consequents'])

        if not antecedents or not consequents:
            continue

        x = antecedents[0]
        y = consequents[0]

        # 买了 X 的客户
        has_x = set(cxp[cxp['产品品种'] == x]['客户编号'].unique())
        # 买了 Y 的客户
        has_y = set(cxp[cxp['产品品种'] == y]['客户编号'].unique())

```



```

# 目标客户：买了 X 但没买 Y
target_customers = has_x - has_y

if target_customers:
    recommendations.append({
        '主力品X': x,
        '互补品Y': y,
        '目标客户数': len(target_customers),
        '目标客户列表': list(target_customers)[:100], # 限制展示
        '综合评分': rule['综合评分'],
        '置信度': rule['confidence'],
        '提升度': rule['lift'],
        '推荐策略': '交叉销售',
    })

result = pd.DataFrame(recommendations)
if len(result) > 0:
    result = result.sort_values('目标客户数', ascending=False)
    logger.info(f"购物篮推荐: {len(result)} 个互补品对")

return result

def run(self) -> pd.DataFrame:
    """执行完整购物篮分析"""
    if not self.config['MARKET_BASKET']['enabled']:
        logger.info("购物篮分析未启用")
        return pd.DataFrame()

    try:
        baskets = self.build_baskets()
        if len(baskets) == 0:
            return pd.DataFrame()

        rules = self.mine_association_rules(baskets)
        if len(rules) == 0:
            return pd.DataFrame()

        scored = self.score_complementary_pairs(rules)
        exp = self.silver['silver_customer_x_product']
        recommendations = self.generate_recommendations(scored, exp)

        return recommendations

    except ImportError:
        logger.error("购物篮分析需要 mlxtend 库: pip install mlxtend")
        return pd.DataFrame()
    except Exception as e:
        logger.error(f"购物篮分析异常: {e}")

```

```
return pd.DataFrame()
```

```
# 第五层：输出标准与 BI 展示（第 6-7 周） {#第五层输出标准与-bi-展示第-6-7-周 }
```

```
## 5.1 Gold 层输出规范 {#51-gold-层输出规范 }
```

| 表名                             | 粒度            | 关键字段                                   | 用途        |
|--------------------------------|---------------|----------------------------------------|-----------|
| gold_customer_profile          | 每客户一行         | 60+ 指标，含经营势能、信用评分、机会/风险分、RFM-π、产品组合健康度 |           |
| gold_customer_x_product        | 客户 × 产品 × 月份  | 月度交易、产品状态、生命周期阶段                       | 下钻桥梁、交叉分析 |
| gold_customer_portfolio_health | 每客户一行         | 各画像产品占比、高风险产品占比、衰退品金额占比                | 产品组合健康    |
| gold_inventory_product         | 每产品一行         | 库存量、周转天数、四分类、备货风险、策略                   | 库存优化      |
| gold_pricing_recommendation    | 销售员 × 客户 × 产品 | 历史均价、建议区间、预计影响、策略类型                    | 报价引导      |
| gold_kpi_daily                 | 每日一行为         | 销售额、数量、订单数、Top5 客户/产品、本月累计             | 准实时监控     |

```
## 5.2 数据时效标识规范 {#52-数据时效标识规范 }
```

| 数据范围     | 标签示例                      | 刷新频率   |
|----------|---------------------------|--------|
| 当日最新 KPI | ""数据截止 2026-05-18 16:00"" | 每日/每小时 |
| 月度分析指标   | ""基于 2026 年 4 月完整数据""     | 月度     |
| 库存数据     | ""库存截至 2026-05-18""       | T+1    |
| 定价建议     | ""基于近 12 月交易数据""          | 月度     |

```
## 5.3 BI 视图设计（五角色融合） {#53-bi-视图设计五角色融合 }
```

```
### 视图 1：销售作战雷达 {#视图-1销售作战雷达 }
```

- \*\*受众\*\*：客户经理、销售总监
- \*\*核心叙事\*\*：谁该跟、谁要救、该报什么价
- \*\*关键元素\*\*：
  - 客户分层（S/A/B/C + KA/AA）分布图
  - 流失预警清单（Top 10）+ 采购中断预警
  - 机会/风险评分 Top 10
  - 客户经营势能四象限矩阵（收入增长率 × 毛利率）
  - 报价建议卡片（历史均价、建议区间、预计影响）
  - 库存预警（该客户常购品的缺货/呆滞提示）
  - 客户 → 产品下钻路径

```
### 视图 2：产品组合看板 {#视图-2产品组合看板 }
```

- \*\*受众\*\*：产品线管理团队
- \*\*核心叙事\*\*：哪些产品健康、哪些在衰退、库存怎么清
- \*\*关键元素\*\*：
  - 九宫格画像分布（气泡图）
  - 高风险产品清单（Top 10）
  - 新品爬坡 Cohort 热力图
  - ASP 趋势和价格偏离度
  - 库存四分类分布（畅销/呆滞可激活/呆滞风险/成长储备）

- 呆滞激活任务清单（阶梯折扣建议、互补品打包方案）
- 缺货预警清单（成长储备型库存覆盖 < 14 天）

### ### 视图 3：信用风控驾驶舱 {#视图-3信用风控驾驶舱 }

- \*\*受众\*\*：财务 / 风控团队
- \*\*核心叙事\*\*：钱收得回来吗、该催谁
- \*\*关键元素\*\*：
  - DSO 趋势图（月度）
  - 信用等级分布（A/B/C/D 占比）
  - 逾期客户清单及逾期天数
  - 回款模式聚类分布
  - 采购中断预警与回款风险交叉表
  - 项目备货客户的回款跟踪

### ### 视图 4：管理层摘要 {#视图-4管理层摘要 }

- \*\*受众\*\*：CEO / VP
- \*\*核心叙事\*\*：本月怎么样了、有什么大事件
- \*\*关键元素\*\*：
  - 上半区：实时 KPI 卡片（今日销售额、本月累计、同比环比、迷你趋势线）
  - 下半区：上月分析摘要（客户分层分布、预警产品数、回款健康度、库存总量变化）
  - 大事件简报（自动扫描）：
    - 重大毛利损失：单客户单产品降价致整体毛利影响 > 1%
    - 备货滞销：近 3 月备货未出库且占某销售员历史销量 > 30%
    - 销售员利润连续下滑：连续 3 月毛利率环比累计降 > 2pp
    - KA/AA 客户利润恶化：环比降 > 3pp 或同比降 > 5pp
  - 库存动态：近 12 月库存增减 Top 10 产品
  - 数据时效标注

### ### 视图 5：分析师工作台 {#视图-5分析师工作台 }

- \*\*受众\*\*：数据分析师
- \*\*核心叙事\*\*：自由探索交叉分析、验证假设
- \*\*关键元素\*\*：
  - 无预设筛选——所有维度可自由组合
  - 支持明细下钻到原始行级数据
  - 购物篮分析结果查询（互补品对、目标客户列表）
  - 降价模拟器（手动调整价格/销量，模拟对毛利率和毛利额的影响）

### ### 视图 6：代理商任务看板 {#视图-6代理商任务看板 }

- \*\*受众\*\*：渠道经理、代理商对接人
- \*\*核心叙事\*\*：代理商能力怎么样、该派什么任务
- \*\*关键元素\*\*：
  - 代理商覆盖评估：KA/AA 客户数、品类渗透率、缺口分析
  - 代理商能力标签：终端激活率、难销品处理能力
  - 任务下发清单：品类导入任务、呆滞激活任务
  - 终端激活率监测：发货至代理商 vs 终端确认收货
  - 渠道集中度风险：Top N 代理商收入占比、单一渠道依赖预警

## ## 5.4 权限隔离实现 {#54-权限隔离实现 }

```
```python
```

```
"""
```

权限控制配置 — 融合文字版三层隔离 + PDF 的 BI 工具无关设计

设计理由:

- BI 工具本身负责行级安全 (Row-Level Security),  
此配置给出每个角色能看到什么、不能看到什么
- `hidden_fields` 中的字段由上游 ETL 脱敏, BI 工具不做二次过滤
- `row_filter` 供 BI 工具配置行级权限使用

```
"""
```

```
ROLE_PERMISSIONS = {
    'sales_rep': {
        'visible_tables': [
            'gold_customer_profile',
            'gold_customer_x_product',
            'gold_pricing_recommendation',
            'gold_kpi_daily',
        ],
        'hidden_fields': ['成本', '毛利率', '毛利额', '标准成本', '盈亏平衡销量', '弹性系数E'],
        'row_filter': "业务负责人 == '{current_user}'",
        'editable': ['反馈按钮', '接受按钮'],
        'views': ['销售作战雷达'],
    },

    'sales_director': {
        'visible_tables': [
            'gold_customer_profile',
            'gold_customer_x_product',
            'gold_pricing_recommendation',
            'gold_kpi_daily',
            'gold_inventory_product',
        ],
        'hidden_fields': [],
        'row_filter': None,
        'editable': ['降价审批', '任务下发'],
        'views': ['销售作战雷达', '信用风控驾驶舱', '管理层摘要（简化版）'],
    },

    'finance': {
        'visible_tables': [
            'gold_customer_profile',
            'gold_kpi_daily',
            'gold_inventory_product',
        ],
        'hidden_fields': ['薪酬', '销售目标'],
        'row_filter': None,
```

```

        'editable': ['信用等级调整', '账期调整'],
        'views': ['信用风控驾驶舱'],
    },

    'product_manager': {
        'visible_tables': [
            'gold_customer_x_product',
            'gold_inventory_product',
            'gold_customer_portfolio_health',
        ],
        'hidden_fields': ['客户回款详情', '信用等级'],
        'row_filter': "产品线 IN '{user_product_lines}'",
        'editable': ['新品标记', '生命周期阶段确认'],
        'views': ['产品组合看板'],
    },

    'ceo_vp': {
        'visible_tables': [
            'gold_customer_profile',
            'gold_kpi_daily',
            'gold_inventory_product',
            'gold_customer_portfolio_health',
        ],
        'hidden_fields': [],
        'row_filter': None,
        'editable': ['战略参数调整'],
        'views': ['管理层摘要'],
    },

    'analyst': {
        'visible_tables': 'ALL',
        'hidden_fields': [],
        'row_filter': None,
        'editable': ['临时分析标记'],
        'views': ['分析师工作台'],
    },
}

```

## 5.5 输出报告自动化（Excel + HTML）

### 设计理由：

- 同时输出 Excel（人肉审阅，透视表）和 HTML（交互可视化）
- 每份报告包含执行摘要，自动扫描关键指标变化
- 执行摘要的参考日期固定（基于配置或数据最大日期）

"""

报告生成器 — 融合两方案的输出格式

输出格式:

- Excel: 多 Sheet 包含明细、透视、矩阵
- HTML: Plotly 交互式图表

新人须知:

- 如果报告内容需要调整, 修改对应 `generate_*` 方法
- HTML 报告可嵌入 BI 工具或邮件发送

"""

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from datetime import datetime
import logging

logger = logging.getLogger(__name__)

class ReportGenerator:
    def __init__(self, gold_tables, config):
        self.gold = gold_tables
        self.config = config

    def generate_data_exploration_report(self):
        """数据探索报告 (PDF 6.2 + 文字版基线)"""
        with pd.ExcelWriter('output/reports/01_data_profile.xlsx', engine='openpyxl') as writer:
            # 客户概况
            profile = self.gold['gold_customer_profile']
            summary = pd.DataFrame({
                '指标': ['客户总数', '直销客户数', '代理客户数', 'KA/AA 客户数'],
                '数值': [
                    len(profile),
                    (profile['渠道类型'] == '直销').sum(),
                    (profile['渠道类型'] == '代理').sum(),
                    profile['客户等级'].isin(['KA', 'AA']).sum(),
                ],
            })
            summary.to_excel(writer, sheet_name='客户概况', index=False)

            # 回款分布
            if 'DS0' in profile.columns:
                payment = profile.agg({
```

```

        'DSO': ['mean', 'median', 'std'],
        '逾期率': ['mean', 'median'],
    })
    payment.to_excel(writer, sheet_name='回款分布')

# 产品概况
if 'gold_inventory_product' in self.gold:
    inv = self.gold['gold_inventory_product']
    product_summary = pd.DataFrame({
        '指标': ['品种总数', '畅销品数', '呆滞风险品数'],
        '数值': [
            inv['产品品种'].nunique(),
            (inv['库存分类'] == '畅销品').sum(),
            (inv['库存分类'] == '呆滞风险型').sum(),
        ],
    })
    product_summary.to_excel(writer, sheet_name='产品概况', index=False)

self._generate_html_data_profile()
logger.info("数据探索报告已生成")

def generate_customer_panorama_report(self):
    """客户全景与分层报告（PDF 6.3 + 文字版分层）"""
    with pd.ExcelWriter('output/reports/02_customer_profile.xlsx', engine='openpyxl') as writer:
        profile = self.gold['gold_customer_profile']
        profile.to_excel(writer, sheet_name='客户全景表', index=False)

        # 集中度分析（帕累托）
        sorted_profile = profile.sort_values('近12月收入', ascending=False).reset_index(drop=True)
        sorted_profile['累计收入占比'] = (
            sorted_profile['近12月收入'].cumsum() / sorted_profile['近12月收入'].sum()
        )
        sorted_profile.to_excel(writer, sheet_name='集中度分析')

self._generate_html_customer_profile()
logger.info("客户全景报告已生成")

def generate_comprehensive_analysis_report(self):
    """综合分析报告"""
    with pd.ExcelWriter('output/reports/03_analysis_report.xlsx', engine='openpyxl') as writer:
        # 盈利矩阵
        cxp = self.gold['gold_customer_x_product']
        margin_matrix = cxp.pivot_table(
            values='毛利率', index='客户编号', columns='产品品种', aggfunc='mean'
        )
        margin_matrix.to_excel(writer, sheet_name='盈利矩阵')

        # 品种生命周期

```

```

        if '状态' in exp.columns:
            lifecycle = exp.groupby(['产品品种', '状态']).size().unstack(fill_value=0)
            lifecycle.to_excel(writer, sheet_name='品种生命周期')

# 机会/风险清单
profile = self.gold['gold_customer_profile']
top_opportunities = profile.nlargest(10, '机会分')[
    ['客户编号', '客户名称', '机会分', '机会等级', '建议动作1']
]
top_opportunities.to_excel(writer, sheet_name='机会清单', index=False)

top_risks = profile.nlargest(10, '风险分')[
    ['客户编号', '客户名称', '风险分', '风险等级', '建议动作1']
]
top_risks.to_excel(writer, sheet_name='风险清单', index=False)

self._generate_html_analysis_report()
logger.info("综合分析报告已生成")

def _generate_html_data_profile(self):
    """HTML 交互版数据探索报告"""
    profile = self.gold['gold_customer_profile']

    fig = make_subplots(rows=2, cols=2,
                        specs=[[{'type': 'domain'}, {'type': 'domain'}],
                              [{'type': 'xy'}, {'type': 'xy'}]])

    channel_dist = profile['渠道类型'].value_counts()
    fig.add_trace(go.Pie(labels=channel_dist.index, values=channel_dist.values), row=1, col=1)

    if 'DS0' in profile.columns:
        fig.add_trace(go.Histogram(x=profile['DS0'], nbinsx=20), row=2, col=1)

    fig.update_layout(title_text='数据探索报告')
    fig.write_html('output/reports/01_data_profile.html')

def _generate_html_customer_profile(self):
    """HTML 交互版客户全景报告"""
    profile = self.gold['gold_customer_profile']

    if '收入同比' in profile.columns and '近12月毛利率' in profile.columns:
        fig = px.scatter(
            profile,
            x='收入同比',
            y='近12月毛利率',
            size='近12月收入',
            color='机会等级',
            hover_data=['客户编号', '客户名称', '风险等级'],

```



```

        title='客户经营势能四象限矩阵',
    )
    fig.add_hline(y=profile['近12月毛利率'].median(), line_dash='dash',
                  annotation_text='毛利率中位')
    fig.add_vline(x=0, line_dash='dash', annotation_text='增长分界线')
    fig.write_html('output/reports/02_customer_profile.html')

def _generate_html_analysis_report(self):
    """HTML 交互版综合分析报告"""
    cxp = self.gold['gold_customer_x_product']
    if '毛利率' in cxp.columns:
        margin_matrix = cxp.pivot_table(
            values='毛利率', index='客户编号', columns='产品品种', aggfunc='mean'
        ).fillna(0)
        fig = px.imshow(
            margin_matrix.iloc[:min(50, len(margin_matrix)), :min(50, margin_matrix.shape[1])],
            labels=dict(x='产品品种', y='客户编号', color='毛利率'),
            title='客户 × 产品盈利矩阵热力图 (Top 50 × 50)',
        )
        fig.write_html('output/reports/03_analysis_report.html')

def run_all(self):
    """执行全部报告生成"""
    self.generate_data_exploration_report()
    self.generate_customer_panorama_report()
    self.generate_comprehensive_analysis_report()
    logger.info("全部报告生成完成: Excel + HTML")

```

# 第六层：准实时 KPI 通道（独立运行）（第 7 周）

- > \*\*设计理由\*\*:
- > - 准实时通道与月度分析解耦，可独立运行
- > - 只做轻量聚合（销售额、订单数、活跃客户等），不做复杂计算
- > - 月度分析完成后，当月累计数据会自动覆盖为月度分析结果
- >
- > 相对原方案的变化:
- > - 保持设计与原方案一致，增加了错误处理
- > - 时效标注固定使用运行时时间，保证可审计

```

```python
"""

```

准实时 KPI 通道 – PDF 方案 4.2 节 + 文字版 T+1 处理

独立于月度分析，每日运行。

输出:

- gold/gold\_kpi\_daily.csv

- gold/kpi\_timestamp.txt (时效标注, 供 BI 工具读取)

```
"""

import pandas as pd
from datetime import datetime
from pathlib import Path
import logging

logger = logging.getLogger(__name__)

class DailyKPIPipeline:
    def __init__(self, config):
        self.config = config
        self.etl_run_id = config.get('ETL_RUN_ID', 'unknown')

    def read_daily_export(self, file_path: str) -> pd.DataFrame:
        """读取最新 ERP 日更数据

        设计理由: T+1 数据, 不需要做工作日调整或复杂清洗,
        只做最少的格式转换(日期解析)。
        """

        try:
            df = pd.read_csv(file_path, parse_dates=['发货日期'])
            logger.info(f"读取日更数据: {len(df)} 行")
            return df
        except FileNotFoundError:
            logger.error(f"日更数据文件未找到: {file_path}")
            raise
        except Exception as e:
            logger.error(f"读取日更数据失败: {e}")
            raise

    def calc_kpi_aggregates(self, df: pd.DataFrame) -> pd.DataFrame:
        """计算每日 KPI 聚合值"""
        # 确定业务日期列
        date_col = '发货日期' if '发货日期' in df.columns else df.columns[0]

        daily = df.groupby(date_col).agg(
            总销售额=('金额', 'sum') if '金额' in df.columns else ('数量', 'sum'),
            总数量=('发货数量', 'sum') if '发货数量' in df.columns else (df.columns[2], 'sum'),
            订单数=('订单编号', 'nunique') if '订单编号' in df.columns else (df.columns[0], 'count')
            活跃客户数=('客户编号', 'nunique') if '客户编号' in df.columns else (df.columns[1], 'nunique')
            活跃品种数=('产品品种', 'nunique') if '产品品种' in df.columns else (df.columns[0], 'nunique')
        ).reset_index()

        daily.columns = ['日期', '总销售额', '总数量', '订单数', '活跃客户数', '活跃品种数']
```

```

# 本月累计
daily['本月'] = daily['日期'].dt.to_period('M')
daily['本月累计销售额'] = daily.groupby('本月')['总销售额'].cumsum()

# 审计字段
daily['etl_run_id'] = self.etl_run_id
daily['updated_at'] = datetime.now().strftime('%Y-%m-%d %H:%M')

return daily

```

```

def export_gold_kpi(self, kpi_df: pd.DataFrame):
    """输出 gold_kpi_daily"""
    gold_dir = Path(self.config.get('GOLD_DIR', 'gold'))
    gold_dir.mkdir(parents=True, exist_ok=True)

    kpi_df.to_csv(gold_dir / 'gold_kpi_daily.csv', index=False)

    # 时效标注
    now_str = datetime.now().strftime('%Y-%m-%d %H:%M')
    with open(gold_dir / 'kpi_timestamp.txt', 'w') as f:
        f.write(f"数据截止 {now_str}")

    logger.info(f"Daily KPI 已更新: {len(kpi_df)} 天数据")

```

```

def run(self, daily_file: str):
    """执行每日 KPI 流程"""
    try:
        df = self.read_daily_export(daily_file)
        kpi = self.calc_kpi_aggregates(df)
        self.export_gold_kpi(kpi)
    except Exception as e:
        logger.error(f"Daily KPI 运行失败: {e}")
        raise

```

# 第七层: Pipeline 编排与运行 (run\_pipeline.py) {#第七层pipeline-编排与运行run\_pipelinepy }

> \*\*设计理由\*\*:

- > - 原方案在目录结构中声明了 `scripts/run\_pipeline.py` 但从未实现
- > - 编排层负责: 确定执行顺序、传递依赖 (表)、判断是否可以继续 (数据质量门禁)
- > - 错误处理: 任何步骤失败时记录状态, 不 panic, 支持重试
- > - Checkpoint: 每步完成后保存中间结果, 重跑时可以从断点恢复

```

```python
"""

```

Pipeline 主编排 - 融合方案的核心执行入口

使用方式:

```
python scripts/run_pipeline.py
```

执行流程：

1. 初始化配置 + 日志
2. 数据质量验证（门禁）
3. Bronze → Silver 转换
4. Gold 层分析（客户、库存、定价）
5. 购物篮分析（可选）
6. 报告生成

新人须知：

- 所有步骤的依赖关系在 `ORCHESTRATION_ORDER` 中定义
- 某一步失败时，后面的步骤不会执行（除非独立运行）
- 查看 `pipeline_log.json` 获取每次运行的详细记录

=====

```
import pandas as pd
import json
import logging
from datetime import datetime
from pathlib import Path

# —— 配置导入 ——
import sys
sys.path.insert(0, str(Path(__file__).parent.parent))
from config import config, ETL_RUN_ID, ETL_RUN_AT

# —— 各引擎导入 ——
from validate_data import DataValidator
from silver_transformer import SilverTransformer
from customer_analysis import CustomerAnalysisEngine
from inventory_analysis import InventoryAnalysisEngine
from pricing_recommendation import PricingRecommendationEngine
from market_basket import MarketBasketAnalyzer
from report_generator import ReportGenerator
from daily_kpi import DailyKPIPipeline

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(name)s] %(levelname)s: %(message)s',
    handlers=[
        logging.FileHandler('logs/pipeline.log'),
        logging.StreamHandler(),
    ],
)

logger = logging.getLogger('pipeline')
```

```
class PipelineOrchestrator:
```

```
    """
```

```
    Pipeline 编排器
```

```
    职责:
```

1. 加载数据
2. 数据质量门禁
3. 按依赖顺序执行各层转换
4. 错误处理与 Checkpoint
5. 运行记录 (pipeline\_log.json)

```
    设计理由:
```

- 单步失败不影响已有结果 (已保存的 `intermediate` 文件可用)
- `checkpoint_dir` 保存每一步的输出路径, 便于排查

```
    """
```

```
def __init__(self, config, data_dir: str = 'data'):
```

```
    self.config = config
```

```
    self.data_dir = Path(data_dir)
```

```
    self.checkpoint_dir = Path('checkpoints')
```

```
    self.checkpoint_dir.mkdir(parents=True, exist_ok=True)
```

```
    self.results = {}
```

```
    self.run_log = {
```

```
        'etl_run_id': config.get('ETL_RUN_ID', 'unknown'),
```

```
        'started_at': config.get('ETL_RUN_AT', datetime.now().isoformat()),
```

```
        'steps': [],
```

```
    }
```

```
def _log_step(self, step_name: str, status: str, detail: str = ''):
```

```
    """记录步骤状态"""
```

```
    self.run_log['steps'].append({
```

```
        'step': step_name,
```

```
        'status': status,
```

```
        'detail': detail,
```

```
        'timestamp': datetime.now().isoformat(),
```

```
    })
```

```
    logger.info(f"[{status}] {step_name}: {detail}")
```

```
def _save_checkpoint(self, name: str, data):
```

```
    """保存中间结果 Checkpoint"""
```

```
    path = self.checkpoint_dir / f'{name}.pkl'
```

```
    if isinstance(data, dict):
```

```
        for k, v in data.items():
```

```
            if isinstance(v, pd.DataFrame):
```

```
                v.to_pickle(self.checkpoint_dir / f'{name}_{k}.pkl')
```

```
    elif isinstance(data, pd.DataFrame):
```

```
        data.to_pickle(path)
```

```

    return path

def _load_checkpoint(self, name: str):
    """加载 Checkpoint (如果存在)"""
    path = self.checkpoint_dir / f'{name}.pkl'
    if path.exists():
        return pd.read_pickle(path)
    return None

def step_01_validate_data(self) -> bool:
    """步骤 1: 数据质量验证 (门禁)"""
    self._log_step('数据质量验证', 'RUNNING')

    try:
        # 加载数据
        source_path = self.data_dir / 'source.xlsx'
        sales_df = pd.read_excel(source_path, sheet_name='销售明细表')
        payment_df = pd.read_excel(source_path, sheet_name='对账回款表') if '对账回款表' in pd.ExcelFile(source_path) else None
        inventory_df = pd.read_excel(source_path, sheet_name='库存表') if '库存表' in pd.ExcelFile(source_path) else None

        # 执行验证
        validator = DataValidator(self.config)
        report = validator.run_all(sales_df, payment_df, inventory_df)

        self.results['validation_report'] = report

        if report['是否可进入分析']:
            self._log_step('数据质量验证', 'PASSED',
                           f"CRITICAL={report['CRITICAL项数']}, WARNING={report['WARNING项数']}")
            return True
        else:
            self._log_step('数据质量验证', 'BLOCKED',
                           f"CRITICAL={report['CRITICAL项数']} > 0, 需修复数据后重试")
            return False

    except Exception as e:
        self._log_step('数据质量验证', 'FAILED', str(e))
        return False

def step_02_silver_transform(self) -> bool:
    """步骤 2: Bronze -> Silver 转换"""
    self._log_step('Silver 层转换', 'RUNNING')

    try:
        transformer = SilverTransformer(self.config)

        # 如果 data 目录有 excel, 先生成 bronze CSV
        source_path = self.data_dir / 'source.xlsx'

```

```

if source_path.exists():
    df_all = pd.read_excel(source_path, sheet_name='销售明细表')
    bronze_dir = Path(self.config.get('BRONZE_DIR', 'bronze'))
    bronze_dir.mkdir(parents=True, exist_ok=True)
    bronze_file = bronze_dir / 'raw_sales_latest.csv'
    df_all.to_csv(bronze_file, index=False)
else:
    bronze_file = Path(self.config.get('BRONZE_DIR', 'bronze')) / 'raw_sales_latest.c

if not bronze_file.exists():
    self._log_step('Silver 层转换', 'FAILED', f'Bronze 文件不存在: {bronze_file}')
    return False

silver_tables = transformer.run(str(bronze_file))
self.results['silver_tables'] = silver_tables
self.results['silver_cleaning_log'] = transformer.cleaning_log

self._log_step('Silver 层转换', 'PASSED',
               f"生成 {len(silver_tables)} 张 Silver 表")
return True

except Exception as e:
    self._log_step('Silver 层转换', 'FAILED', str(e))
    return False

def step_03_gold_analysis(self) -> bool:
    """步骤 3: Gold 层分析（客户 + 库存 + 定价）"""
    self._log_step('Gold 层分析', 'RUNNING')

    try:
        silver = self.results.get('silver_tables')
        if not silver:
            self._log_step('Gold 层分析', 'SKIPPED', '缺少 Silver 表')
            return False

        # 3a. 客户分析
        self._log_step('客户分析引擎', 'RUNNING')
        customer_engine = CustomerAnalysisEngine(self.config, silver)
        customer_profile = customer_engine.run()
        self.results['gold_customer_profile'] = customer_profile
        customer_profile.to_csv('gold/gold_customer_profile.csv', index=False)
        self._log_step('客户分析引擎', 'PASSED', f"{len(customer_profile)} 条")

        # 3b. 库存分析
        self._log_step('库存分析引擎', 'RUNNING')
        inventory_path = self.data_dir / 'source.xlsx'
        inventory_df = pd.read_excel(inventory_path, sheet_name='库存表') if inventory_path.e
        if len(inventory_df) > 0:

```

```

        inv_engine = InventoryAnalysisEngine(self.config, inventory_df, silver)
        inv_profile = inv_engine.run()
        self.results['gold_inventory_product'] = inv_profile
        inv_profile.to_csv('gold/gold_inventory_product.csv', index=False)
        self._log_step('库存分析引擎', 'PASSED', f"{len(inv_profile)} 个产品")
    else:
        self._log_step('库存分析引擎', 'SKIPPED', '无库存数据')

# 3c. 定价建议
self._log_step('定价建议引擎', 'RUNNING')
cost_data = pd.read_excel(inventory_path, sheet_name='成本表') if inventory_path.exists()
pricing_engine = PricingRecommendationEngine(self.config, silver, cost_data)
sales_rep_view, director_view = pricing_engine.run()
self.results['sales_rep_view'] = sales_rep_view
self.results['director_view'] = director_view
if len(sales_rep_view) > 0:
    sales_rep_view.to_csv('gold/gold_pricing_recommendation.csv', index=False)
self._log_step('定价建议引擎', 'PASSED',
               f"销售员版 {len(sales_rep_view)} 条")

self._log_step('Gold 层分析', 'PASSED')
return True

except Exception as e:
    self._log_step('Gold 层分析', 'FAILED', str(e))
    return False

def step_04_market_basket(self) -> bool:
    """步骤 4: 购物篮分析（可选）"""
    if not self.config['MARKET_BASKET']['enabled']:
        self._log_step('购物篮分析', 'SKIPPED', '未启用')
        return True

    self._log_step('购物篮分析', 'RUNNING')

    try:
        silver = self.results.get('silver_tables')
        if not silver:
            self._log_step('购物篮分析', 'SKIPPED', '缺少 Silver 表')
            return False

        basket_engine = MarketBasketAnalyzer(self.config, silver)
        recommendations = basket_engine.run()
        self.results['basket_recommendations'] = recommendations

        if len(recommendations) > 0:
            recommendations.to_csv('gold/gold_basket_recommendations.csv', index=False)
            self._log_step('购物篮分析', 'PASSED',

```



```

        f"{len(recommendations)} 个互补品对")
    else:
        self._log_step('购物篮分析', 'PASSED', '无推荐结果')
    return True

except Exception as e:
    self._log_step('购物篮分析', 'FAILED', str(e))
    return False

def step_05_generate_reports(self) -> bool:
    """步骤 5: 报告生成"""
    self._log_step('报告生成', 'RUNNING')

    try:
        # 收集 Gold 表
        gold_tables = {
            'gold_customer_profile': self.results.get('gold_customer_profile', pd.DataFrame()),
            'gold_inventory_product': self.results.get('gold_inventory_product', pd.DataFrame()),
            'gold_pricing_recommendation': self.results.get('sales_rep_view', pd.DataFrame()),
            'gold_customer_x_product': self.results.get('silver_tables', {}).get(
                'silver_customer_x_product', pd.DataFrame()
            ),
        }

        reporter = ReportGenerator(gold_tables, self.config)
        reporter.run_all()

        self._log_step('报告生成', 'PASSED')
        return True

    except Exception as e:
        self._log_step('报告生成', 'FAILED', str(e))
        return False

def run(self):
    """执行完整 Pipeline

    执行顺序:
    1. 数据质量验证（必须通过才能继续）
    2. Silver 层（必须成功才能继续）
    3. Gold 层分析
    4. 购物篮（可选，失败不阻断）
    5. 报告生成
    """
    logger.info(f"=" * 60)
    logger.info(f"Pipeline 启动 | ETL_RUN_ID={ETL_RUN_ID}")
    logger.info(f"=" * 60)

```

```
# 步骤 1: 数据质量验证 (门禁)
if not self.step_01_validate_data():
    self.run_log['final_status'] = 'BLOCKED_BY_DATA_QUALITY'
    self._save_final_log()
    logger.error("Pipeline 阻断: 数据质量未达标")
    return
```

```
# 步骤 2: Silver 层
if not self.step_02_silver_transform():
    self.run_log['final_status'] = 'FAILED_AT_SILVER'
    self._save_final_log()
    logger.error("Pipeline 失败: Silver 层转换错误")
    return
```

```
# 步骤 3: Gold 层
gold_ok = self.step_03_gold_analysis()
if not gold_ok:
    self.run_log['final_status'] = 'FAILED_AT_GOLD'
    self._save_final_log()
    logger.error("Pipeline 失败: Gold 层分析错误")
    return
```

```
# 步骤 4: 购物篮 (可选)
self.step_04_market_basket()
```

```
# 步骤 5: 报告生成
self.step_05_generate_reports()

self.run_log['final_status'] = 'COMPLETED'
self._save_final_log()
logger.info(f"=" * 60)
logger.info(f"Pipeline 完成 | ETL_RUN_ID={ETL_RUN_ID}")
logger.info(f"=" * 60)
```

```
def _save_final_log(self):
    """保存运行日志"""
    self.run_log['finished_at'] = datetime.now().isoformat()
    log_path = Path('logs') / f'pipeline_{ETL_RUN_ID}.json'
    log_path.parent.mkdir(parents=True, exist_ok=True)
    with open(log_path, 'w', encoding='utf-8') as f:
        json.dump(self.run_log, f, ensure_ascii=False, indent=2)
    logger.info(f"运行日志已保存: {log_path}")
```

```
# —— 独立运行 ——
```

```
if __name__ == '__main__':
    orchestrator = PipelineOrchestrator(config)
    orchestrator.run()
```

## 7.1 独立运行 KPI 通道

```
```python
# run_kpi_daily.py - 准实时 KPI 通道，独立于主 Pipeline 运行 {#run_kpi_daily.py--准实时-kpi-通道独立于主
# 使用方式: python scripts/run_kpi_daily.py {#使用方式-python-scriptsrun_kpi_daily.py }

from config import config
from daily_kpi import DailyKPIPipeline

pipeline = DailyKPIPipeline(config)
pipeline.run('data/daily_export.csv')
```

# 第八层：测试与验证策略

设计理由：

- 原方案完全未涉及测试，这是生产级系统的必要组成部分
- 核心清洗逻辑（退货匹配、买赠还原、工作日调整）需要单元测试覆盖
- 验收测试确保 Gold 层输出被业务方认可

## 8.1 单元测试

| 测试目标                     | 测试内容     | 测试方法              | 预期结果              |
|--------------------------|----------|-------------------|-------------------|
| match_returns            | 后进先出退货匹配 | 构造 3 笔正交易 + 1 笔退货 | 退货匹配到最近的正交易       |
| apply_workday_adjustment | 工作日系数计算  | 构造 1 月/2 月/3 月数据  | 2 月调整系数 > 1.0     |
| allocate_promotion       | 买赠数量比例分摊 | 构造 2 产品组（买一送一）    | 等效净单价一致           |
| flag_project_stock       | 项目备货标记   | 构造单次采购量 > 3× 月均   | 标记为项目备货           |
| classify_inventory       | 库存四分类    | 构造符合各分类条件的数据      | 正确分到对应类别          |
| calc_markdown_strategy   | 盈亏平衡计算   | 构造数据集后手动计算        | 目标销量 ≥ 1.2× 盈亏平衡  |
| _fit_elasticity          | 弹性系数拟合   | 构造 5 次价格变动的时序数据   | 拟合值在 [-5, -0.1] 内 |

# 单元测试示例

```
# tests/test_silver_transformer.py {#teststest_silver_transformer.py }
"""SilverTransformer 核心清洗逻辑的单元测试"""

import pytest
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from silver_transformer import SilverTransformer

class TestMatchReturns:
    """退货匹配 LIFO 逻辑测试"""

    def setup_method(self):
        self.config = {
            'COLUMN_MAPPING': {
                '客户编号': ['客户编号'],
                '产品品种': ['产品品种'],
                '发货日期': ['发货日期'],
                '发货数量': ['发货数量'],
                '金额': ['金额'],
                '成本': ['成本'],
            },
            'CLEANING_RULES': {
                'return_matching': {
                    'match_window_days': 90,
                },
            },
            'ETL_RUN_ID': 'test',
        }

    def test_lifo_matching(self):
        """测试后进先出：退货应匹配最近的销售"""
        transformer = SilverTransformer(self.config)

        # 构造：客户 A 在 1/1、2/1、3/1 各有 10 的销售，4/1 退货 15
        df = pd.DataFrame({
            '客户编号': ['A', 'A', 'A', 'A'],
            '产品品种': ['P1', 'P1', 'P1', 'P1'],
            '发货日期': pd.to_datetime(['2026-01-01', '2026-02-01', '2026-03-01', '2026-04-01']),
            '发货数量': [10, 10, 10, -15],
            '金额': [100, 100, 100, -150],
            '成本': [60, 60, 60, -90],
        })
```

```
result = transformer.match_returns(df)

# 退货 15 应匹配最近的 10 (3/1) + 5 (2/1)
assert result.loc[3, '净数量'] == -15 # 总退货
# 正向记录的净数量不应改变
assert result.loc[0, '净数量'] == 10 # 1/1 的未受影响
assert result.loc[2, '已匹配'] == True

def test_no_match_outside_window(self):
    """超过 90 天窗口的退货不应匹配"""
    transformer = SilverTransformer(self.config)

    df = pd.DataFrame({
        '客户编号': ['A', 'A'],
        '产品品种': ['P1', 'P1'],
        '发货日期': pd.to_datetime(['2025-01-01', '2026-04-01']),
        '发货数量': [10, -15],
        '金额': [100, -150],
        '成本': [60, -90],
    })

    result = transformer.match_returns(df)
    # 退货超过 90 天，不应匹配
    assert result.loc[1, '已匹配'] == True # 不会有 KeyError
```

8.2 集成测试

| 测试场景               | 数据规模                  | 验证方法                     |
|--------------------|-----------------------|--------------------------|
| 端到端 Pipeline（全部步骤） | 100 客户 × 50 产品 × 24 月 | Gold 输出完整性 + 无报错         |
| 极端场景：全是退货          | 10 客户 × 100% 退货       | Pipeline 不崩溃             |
| 极端场景：空数据           | 无销售记录                 | 优雅降级（空 DataFrame）        |
| 数据质量阻断场景           | 客户编号全部为空              | validate_data 返回 BLOCKED |

8.3 验收测试（业务方参与）

| 验收标准          | 验证方法                  | 责任人  |
|---------------|-----------------------|------|
| 客户分层结果与业务认知一致 | 随机抽 20 个客户，业务判断分层是否合理 | 销售总监 |
| 库存分类与仓库实际一致   | 抽 10 个产品对比分类和实物状态     | 仓库经理 |
| 定价建议的提价空间是否可行 | 抽 5 个提价建议，与客户经理确认     | 销售经理 |

| 验收标准           | 验证方法            | 责任人  |
|----------------|-----------------|------|
| 购物篮互补品对是否有业务意义 | 抽 3 个推荐对，产品经理判断 | 产品经理 |

## 8.4 运行验证（自动化）

每次 Pipeline 运行后，自动执行以下检查：

```
# tests/test_pipeline_output.py {#teststest_pipeline_outputpy }
"""Pipeline 输出一致性检查"""

def test_gold_tables_have_required_columns():
    """所有 Gold 表必须包含 etl_run_id 和 created_at"""
    required_audit = ['etl_run_id', 'created_at']
    for table_name in ['gold_customer_profile', 'gold_inventory_product']:
        df = pd.read_csv(f'gold/{table_name}.csv')
        for col in required_audit:
            assert col in df.columns, f"{table_name} 缺少 {col}"

def test_gold_customer_profile_integrity():
    """客户画像表完整性"""
    df = pd.read_csv('gold/gold_customer_profile.csv')
    assert df['客户编号'].is_unique, "客户编号应唯一"
    assert (df['近12月收入'] >= 0).all(), "收入不应为负"
    assert df['机会分'].between(0, 1).all(), "机会分应在 [0, 1]"
    assert df['风险分'].between(0, 1).all(), "风险分应在 [0, 1]"

def test_inventory_classification_complete():
    """所有产品应有库存分类"""
    df = pd.read_csv('gold/gold_inventory_product.csv')
    assert df['库存分类'].notna().all(), "库存分类不应为空"
    assert df['库存分类'].isin([
        '畅销品', '呆滞可激活型', '呆滞风险型', '成长储备型', '其他'
    ]).all(), "库存分类应在预期范围内"
```

# 实施时间线（融合修订版）

| 阶段  | 内容                  | 周期    | 输出                                             | 验收标准           |
|-----|---------------------|-------|------------------------------------------------|----------------|
| 阶段零 | 数据质量验证 + 环境搭建       | 3-5 天 | 数据质量报告、参数确认                                    | CRITICAL 项数为 0 |
| 阶段一 | 共享数据管道（Silver 层）    | 3-5 天 | silver_*.csv + 清洗日志                            | 与手工核对一致率 > 95% |
| 阶段二 | 客户分析核心引擎 + 库存分析     | 5-7 天 | gold_customer_profile + gold_inventory_product | 分层结果业务认可       |
| 阶段三 | 定价建议 + 购物篮分析        | 3-5 天 | gold_pricing_recommendation + 互补品推荐            | 推荐转化率 > 15%    |
| 阶段四 | 准实时 KPI 通道（与阶段一并行）  | 2-3 天 | gold_kpi_daily + 定时任务                          | 与月度数据差异 < 1%   |
| 阶段五 | BI 接入 + 权限配置        | 3-5 天 | 5+1 角色视图 + 下钻路径                                | 业务用户验收通过       |
| 阶段六 | 单元测试 + 集成测试（与阶段二并行） | 3-5 天 | 测试报告、覆盖率 > 70%                                 | 核心逻辑测试通过       |
| 阶段七 | 试运行 + 反馈调优          | 2-3 周 | 参数调整 + 阈值优化                                    | 误报率 < 20%      |

总计预估：5-7 周核心功能 + 2-3 周试运行调优

注意：时间线相比原方案延长了 2-3 周，原因是：

- 1. 原方案 3-4 周过于乐观，忽略了测试和数据治理的工时
- 2. 弹性和评分逻辑需要更多时间验证业务合理性
- 3. 准实时 KPI 通道提前与阶段一并行，不额外占用时间

## 关键成功条件与失效场景

### 成功条件

| 条件     | 说明                            | 验证方式                |
|--------|-------------------------------|---------------------|
| 数据质量红线 | 客户编号空值/重复率 < 1%，渠道类型准确率 > 95% | validate_data.py 报告 |

| 条件       | 说明                                    | 验证方式              |
|----------|---------------------------------------|-------------------|
| 团队技能     | 至少 1 人具备 Python pandas 能力，1 人熟悉 BI 工具 | 技能矩阵确认            |
| ERP 导出能力 | 支持每日自动导出，或至少月度批量导出                    | 技术对接确认            |
| 业务确认     | 样品单阈值、回款账期、评分分位切割点需业务签字确认             | 会议纪要              |
| 节假日数据    | 提供准确的节假日历（影响工作日调整的准确性）                | holidays.xlsx 完整性 |

## 失效场景与降级方案

| 失效场景                  | 后果          | 降级方案                                                                                                               |
|-----------------------|-------------|--------------------------------------------------------------------------------------------------------------------|
| 数据质量 CRITICAL 项无法清零   | Pipeline 阻断 | <ul style="list-style-type: none"><li>• 退回数据治理，延期 2-3 个月</li><li>• 如果仅是部分客户数据问题，使用 allowlist 排除后继续</li></ul>       |
| 团队无 Python 技能且拒绝学习    | 方案落地困难      | <ul style="list-style-type: none"><li>• 降级为 Excel VBA 方案</li><li>• 购物篮分析和降价模拟大幅缩水</li><li>• 工作日调整改用预设系数表</li></ul> |
| ERP 完全不支持自动化导出        | 准实时通道失效     | <ul style="list-style-type: none"><li>• 退化为纯月度分析</li><li>• 增加手动导出模板 + 批处理脚本</li></ul>                              |
| 代理商拒绝任何数据共享           | 渠道穿透分析失效    | <ul style="list-style-type: none"><li>• 仅能基于出货端数据</li><li>• 用代理商名称+发货数据推断终端信息</li></ul>                            |
| 购物篮分析无频繁项集<br>（数据太稀疏） | 无推荐结果       | <ul style="list-style-type: none"><li>• 降低 min_support（从 0.01 → 0.005）</li><li>• 改为只使用订单篮子和时间篮子</li></ul>          |
| 价格弹性系数无法拟合            | 降价模拟不准确     | <ul style="list-style-type: none"><li>• 使用行业标准弹性系数</li><li>• 或改为纯规则判断（不依赖弹性）</li></ul>                             |
| 试运行误报率 > 20%          | 业务方失去信任     | <ul style="list-style-type: none"><li>• 收紧预警阈值</li><li>• 增加人工复核环节（预警→确认→推送）</li></ul>                              |